CERTIPROOF — COMPLETE PROJECT CONTEXT REPORT

CertiProof
October 16, 2026

CONTENTS

CERTIPROOF — COMPLETE PROJECT CONTEXT REPORT

Evidence tiers used throughout

0. Executive Summary

1. Project Identity

1.1 Titles (as recorded — multiple forms exist)

1.2 One-paragraph overview

1.3 Problem being solved

1.4 Why existing academic certificate systems are weak (as evidenced in this project)

1.5 Motivation for each technology (as implemented, not aspirational)

1.6 Target users

1.7 Main objective

1.8 Specific objectives (derived from what is actually implemented)

1.9 Core contribution of the project

1.10 What makes CertiProof different from a normal certificate database

2. Research Problem

2.1 Problem statement

2.2 Existing-system limitations (evidence-based)

2.3 Security problems

2.4 Trust problems

2.5 Centralization problems

2.6 Certificate forgery / tampering problems

2.7 Verification problems

2.8 Availability / dependency problems

2.9 Data integrity problems

2.10 Metadata poisoning risks

2.11 Why conventional PDF certificates alone are insufficient

A. Research gap

B. Research questions

C. Research objectives

D. Proposed hypothesis

3. Research Gap

4. Objectives

4.1 Main objective

CONFIDENTIAL

4.2 Specific objectives (implementation-mapped)
4.3 Deliverables actually present in the repository
4.4 Deliverables planned but absent
5. System Architecture
5.1 Component inventory
5.2 Stack (resolved from package.json / bun.lock)
5.3 Trust-boundary diagram (textual)
5.4 Component notes worth carrying into the paper
6. End-to-End Issuance
6.1 Exact stage sequence
6.2 What is stored where
6.3 Transaction, confirmation, receipt
6.4 Wrong network behaviour
6.5 Failure modes summary
7. Cryptographic Fingerprint Protocol
7.1 Inputs, ordering, canonicalization
7.2 Pseudocode (derived literally from the implementation)
7.3 Protocol vector (from tests)
7.4 Storage and recomputation
7.5 The three hashes — definitions and trust relationship
7.6 Behaviour if one field changes
7.7 Known protocol properties / caveats
8. Smart Contract
8.1 Identification
8.2 Struct
8.3 State, mappings, arrays
8.4 Events
8.5 Modifiers and functions
8.6 State transitions
8.7 What is and is not permanently anchored
8.8 Why the blockchain is authoritative for each fact
8.9 Gas
8.10 Security properties and limitations visible in code
9. IPFS / Pinata Architecture
9.1 Flow

CONFIDENTIAL

9.2 Validation rules (exact)
9.3 Credential handling and the server/client boundary
9.4 Error handling (complete set)
9.5 Retrieval, integrity, deduplication
9.6 Benchmark evidence — benchmarks/ipfs-results.json
10. Supabase Architecture
10.1 Client and schema
10.2 Operations
10.3 Authority split
10.4 The metadata poisoning problem (Task 9)
10.5 RLS status — strict verdict
11. Verification Method A — verifyCertificate(hash)
11.1 Exact sequence
11.2 Checks, in exact order
11.3 Status is chain-only
12. Verification Method B — QR
12.1 Payload
12.2 Extraction
12.3 After extraction
12.4 Scope limit
13. Verification Method C — document upload (analyzeDocument)
13.1 Flow
13.2 What each method proves — comparison table
14. PDF Extraction and Tamper Detection (Task 7)
14.1 What is written into the PDF
14.2 Why the original implementation failed
14.3 How the current extractor works
14.4 Tamper cases — offline test matrix (test/method-c-extraction.test.ts, 14 tests)
14.5 Live Method C results
14.6 What Method C can and cannot detect
15. Transaction Confirmation State Machine (Task 4)
15.1 States and reason codes
15.2 Receipt validation rules
15.3 Why the old implementation was unsafe
15.4 Timeout, final check, replacement, cleanup

CONFIDENTIAL

15.5 Pending registry and one-shot re-check
15.6 The 29 confirmation tests
16. Wrong-Network Protection (Task 8)
16.1 The original vulnerability
16.2 The guard
16.3 Why the guard must precede fingerprint/PDF/IPFS/signing
16.4 Test result
17. Supabase Integrity Hardening (Task 9)
17.1 The seven protections
17.2 The re-keying attack
17.3 The poisoned-status attack
17.4 Chain-only behaviour and RPC-error behaviour
17.5 Why blockchain stays authoritative even with malicious metadata
18. Security / Threat Model
19. Experimental Setup
19.1 Environment (as recorded in the benchmark JSONs)
19.2 Harness design
19.3 Timing boundaries (as stored in the JSONs)
19.4 Statistical conventions
19.5 What is NOT recorded
20. Performance Results
20.1 Issuance — benchmarks/issuance-results.json (generated 2026-10-16T08:11:24.156Z)
20.2 Verification — benchmarks/verification-results.json (generated 2026-10-15T17:36:44.832Z)
20.3 IPFS — benchmarks/ipfs-results.json
20.4 Unified performance table
20.5 Accuracy / correctness figures
21. Live Experiments
22. Testing Evidence
22.1 Framework and command
22.2 Complete test inventory — 123 tests in 7 files, 28 describe blocks, 0 it() calls
22.3 What each group covers
22.4 Hermeticity
22.5 Gates recorded per task
22.6 Flaky runs — accurate characterisation
22.7 Typecheck / lint / build

CONFIDENTIAL

22.8 Security scanning

23. Final System Properties

24. Limitations

25. Future Work

A. Engineering improvements

B. Security improvements

C. Research extensions

D. Scalability improvements

E. Deployment improvements

26. IEEE Paper Contributions

C1 (strongest) — A chain-authoritative verification architecture that remains correct against a malicious metadata store

C2 — A canonical metadata fingerprint protocol with strict hash-domain separation, proven backward compatible

C3 — Document re-derivation: verifying a presented PDF by recomputing the registry fingerprint from its own printed fields

C4 — Honest transaction outcome semantics (pending as a first-class non-success state)

C5 — Pre-signing network validation preventing no-op “successful” transactions

C6 (weakest) — Reproducible measurement of an integrated issuance/verification pipeline on a public testnet

Claims that MUST be citation-supported in an IEEE paper

27. Safe Claims vs Unsupported Claims

A. Claims strongly supported by implementation + experiments

B. Claims that require citations or literature comparison

C. Claims we MUST NOT make (with the exact overclaim flagged)

28. Required Figures

Fig. 1 — System Architecture

Fig. 2 — Certificate Issuance Workflow

Fig. 3 — Verification Workflow A / B / C

Fig. 4 — Blockchain / IPFS / Supabase Data Model

Fig. 5 — Fingerprint Generation and Verification

Fig. 6 — Transaction Confirmation State Machine

Fig. 7 — Tamper Detection Workflow (Method C)

Fig. 8 — Performance Results

29. Required Tables

30. Literature Review Requirements

31. 30-Second Explanation

32. 1-Minute Explanation

33. 3-Minute Explanation

34. IEEE-Style Technical Summary

35. Final Consistency Audit

Discrepancies (explicit)

PAPER-READY FACT SHEET

DO NOT INVENT

Unknown / not in the repository

Externally configured dependencies (cannot be completed from this repo)

Insufficiently measured — do not publish as results

Claims that must not be made

CERTIPROOF — COMPLETE PROJECT CONTEXT REPORT

Purpose: factual, 0→100 project context for an AI / research assistant that will write an IEEE-quality research paper.

Source of authority: the repository working tree at `C:\Users\iamro\OneDrive\Desktop\MyProjects\Final-year-project\Trust Certificates` (branch `main`), plus the session history in `context_md/`. Every claim below is traceable to a file, a line number, a stored JSON path, or a dated session note.

Hard rules observed while producing this report:

- No code was modified, no feature was created, no fix was applied, no destructive command was run, no transaction was sent.
- Nothing was invented. Where a value is unknown it is written `UNKNOWN` / `NOT IN REPO` / `NOT MEASURED`.
- Where two files disagree, the disagreement is reported, not silently resolved.

Evidence tiers used throughout

Tag	Meaning
[IV]	Implemented and verified (code inspected + gated by tests/typecheck/build or a recorded live run)
[INV]	Implemented but not fully verified (code exists; no test or experiment proves the end-to-end behaviour)
[EM]	Experimentally measured (a stored benchmark JSON or a dated live run records the number)
[OL]	Observed limitation (a real defect, gap, or constraint seen in code or measurement)
[FW]	Future work (not implemented)
[EXT]	External configuration dependency (cannot be done or verified from this repository)

---

0. Executive Summary

CertiProof is a final-year academic project: a web platform for issuing, revoking, and verifying academic certificates whose authenticity is anchored on the Polygon Amoy testnet. An issuer connects MetaMask, fills a certificate form, and the client computes a SHA-256 **metadata fingerprint** over a canonical string, renders a PDF with jsPDF, pins the PDF to IPFS through Pinata, and calls `registerCertificate(...)` on a deployed Solidity contract `CertChainRegistry`. A verifier — with no wallet and no gas — can check a certificate three ways: **Method A** (paste the 0x-hashed fingerprint), **Method B** (scan a QR code that encodes the same verification URL), and **Method C** (upload the PDF, whose printed fields are re-extracted and re-hashed and compared against the on-chain anchor). [IV]

The distinguishing design decision is an explicit **trust hierarchy**: the blockchain is authoritative for certificate identity (`certificate_id` + `certificate_hash`), issuer, status, and timestamp; Supabase is an untrusted display/index layer whose rows are reconciled against the chain and dropped when they do not anchor; IPFS holds document bytes addressed by CID; the QR code is only a transport for the fingerprint. [IV]

Ten staged tasks produced a measured evidence base: fingerprint protocol consolidation (Task 1), server-side Pinata credential handling (Task 2), issuance instrumentation with a 10-run testnet baseline (Task 3), a real three-state transaction confirmation state machine replacing a fabricated-receipt success path (Task 4), an IPFS reliability/integrity/performance benchmark (Task 5), a verification-pipeline benchmark (Task 6), a rewritten Method C PDF extractor that made recipient/course tampering detectable (Task 7), a security and production-readiness audit adding wrong-network guards (Task 8), Supabase↔chain integrity hardening (Task 9), and a final end-to-end live verification of the whole lifecycle (Task 10). [IV] [EM]

The current test suite is **123 tests in 7 files, all offline/hermetic**, passing with `bun test`, alongside `tsc --noEmit` and a production `vite build`. [IV]

Honest headline for the paper: the system's strongest contributions are (a) a precisely specified and backward-compatible metadata fingerprint protocol with strict domain separation from the document byte hash, (b) a chain-authoritative verification design that survives a malicious metadata store, and (c) a documented, reproducible tamper-detection and confirmation-safety evaluation on a public testnet. The weakest areas are scalability (linear O(n) on-chain scan dominating verification latency), external dependencies (Supabase RLS not applied, unauthenticated IPFS upload, free public RPC), and sample sizes (several measurements are small-n or single-run). [EM] [OL] [EXT]

---

1. Project Identity

1.1 TITLES (AS RECORDED — MULTIPLE FORMS EXIST)

Form	Where it appears
CertiProof (project name)	`CertiProof_Master_Build.md:36`, `context_md/audit.md:9`
Blockchain-Based Secure Certificate Verification (stated project title)	`CertiProof_Master_Build.md:40`
CertiProof — Blockchain-Based Secure Academic Certificate Issuance and Verification Platform (full descriptive title)	`context_md/audit.md:9`
CertiProof — Blockchain-Based Secure Certificate Platform (AGENTS.md framing)	`AGENTS.md`

Recommended canonical form for the paper: **"CertiProof: Blockchain-Based Secure Academic Certificate Issuance and Verification Platform."** This is the only form that states both halves of the system (issuance *and* verification) and matches the audit baseline. [IV]

1.2 ONE-PARAGRAPH OVERVIEW

CertiProof is a full-stack web application that lets an authorized institutional wallet issue a digital academic certificate and lets any third party independently verify it. On issuance the client canonicalises four certificate fields (`certificateId | recipientName | course | issueDate`), computes SHA-256 over the UTF-8 canonical string, and uses the resulting `0x`-prefixed 64-character lowercase hex digest as the certificate's identity — the **metadata fingerprint**. That fingerprint, the IPFS CID of the generated PDF, the certificate number, the issuer address, and a block timestamp are written to the `CertChainRegistry` smart contract on Polygon Amoy; the human-readable metadata and transaction hash are written to a Supabase `certificates` row after a real mined receipt exists; the PDF bytes are pinned to IPFS via Pinata through a server-only function so the JWT never reaches the browser. Verification is wallet-free and read-only, offered as three methods (hash entry, QR scan, document upload) whose outcomes are computed against the on-chain record, never against the database alone. [IV]

1.3 PROBLEM BEING SOLVED

From `CertiProof_Master_Build.md:47-61`, traditional academic certificate verification is: slow; manual; dependent on contacting the issuing institution; centralized; vulnerable to document tampering; difficult to scale across institutions; and difficult for employers or third parties to verify quickly. The project's aim, in the spec's own words, is "a trusted digital certificate ecosystem in which authorized institutions can issue certificates and external parties can independently verify their authenticity." [IV]

1.4 WHY EXISTING ACADEMIC CERTIFICATE SYSTEMS ARE WEAK (AS EVIDENCED IN THIS PROJECT)

Weakness	Concrete evidence in this project
Verification requires contacting the issuer	`CertiProof_Master_Build.md:49-55` lists “Dependent on contacting the issuing institution” as a core problem
A PDF alone proves nothing	`context_md/audit.md:78-100` (C-03/C-04): a plain PDF’s bytes are not authenticated, so a copied-and-edited PDF is indistinguishable from a genuine one without an external anchor
Centralized databases are mutable	`context_md/2026-10-16_132041.md:29-37` documents the concrete risk that a mutable metadata store can render fake recipient/status/transaction data as “Verified”
No tamper evidence	Task 6 measured that recipient/course tampering in a PDF went undetected (`2/4` detection) before the Task 7 extractor rewrite — an empirical demonstration of the weakness, not a rhetorical claim [EM]
Third parties cannot check status at scale	On-chain revocation gives an immediate `REVOKED` / `ACTIVE` answer without querying the institution (`contracts/CertChainRegistry.sol:151-163`)

1.5 MOTIVATION FOR EACH TECHNOLOGY (AS IMPLEMENTED, NOT ASPIRATIONAL)

Blockchain (Polygon Amoy). Chosen as the single authoritative, append-only record of *existence, hash, issuer, status, and issue timestamp*. The code states this directly: *”The blockchain is authoritative for certificate identity (certificate_id + certificate_hash) and status”* (`src/lib/certificates.ts:274-281`). Because the contract is public and permissionless to read, any verifier can answer “does this certificate exist, was it issued by an authorized wallet, and is it revoked?” without the institution’s cooperation and without owning gas (`src/lib/blockchain/metamask.ts:46-52` read-only provider “Does NOT require MetaMask. Works for any visitor”). Testnet (Amoy, chain ID 80002) is mandated so no real funds are spent (`CertiProof_Master_Build.md:119-124`). [IV]

Cryptographic hashing (SHA-256). Motivation is deterministic, order-fixed, collision-resistant identity for a *small, canonical* payload rather than for a mutable file. The protocol is deliberately canonical (fixed field order, fixed separator, trim rules, date normalisation) so that the same logical certificate always hashes identically across issuance, verification, and re-derivation from a PDF — proven by 20 dedicated tests including a protocol vector against an independent WebCrypto oracle and legacy-formula equivalence (`test/certificate-fingerprint.test.ts`). [IV]

IPFS (via Pinata). Motivation is content-addressed document storage: the CID is deterministic over content, so identical bytes collapse to one CID and any retrieved copy can be byte-compared to the original. This was verified empirically rather than assumed — same file uploaded twice produced the *same* CID, a different file produced a different CID (`benchmarks/ipfs-results.json` → `repeatedUploads.sameFileCidEqual = true` , `differentFileCidEqual = false`), and 36/36 retrievals were byte-identical to their originals (`reliability.integrity` = `36/36 (100%)`). [EM]

QR-based verification. Motivation is removing friction and copy/paste error from the verifier’s path: the QR encodes `https://certiproof.app/verify/<fingerprint>` (`src/components/certificate/CertificatePreview.tsx:277`), so a phone camera yields exactly the input Method A expects. Test 55–63 of `test/verification.test.ts` proves a QR-extracted hash flows through the identical validation pipeline as a hand-entered one. **Important scope limit:** there is no in-app camera scanner; scanning is expected to happen with an external camera app (`src/routes/verify.tsx:144-168` is a descriptive placeholder). [IV] [OL]

1.6 TARGET USERS

User	How the system addresses them	Evidence
Educational institutions	Issuer login + dashboard + bulk CSV issuance + organization settings	`src/routes/login.tsx`, `dashboard.*.tsx`, `issue_.bulk.tsx`, `settings.tsx` [IV]
Authorized issuers	Wallet must be in the contract's `authorizedIssuers` mapping to register; login gate calls `checkIssuerAuth(address)`	`contracts/CertChainRegistry.sol:39-42`, `src/routes/login.tsx:48-59` [IV]
Students / credential holders	Receive the PDF + QR; the certificate detail page is public and shows blockchain proof	`src/routes/certificate.$id.tsx` (public) [IV]
Employers / verifiers	Wallet-free Methods A/B/C on `/verify`	`src/routes/verify.tsx`, `verify_.$hash.tsx` [IV]

1.7 MAIN OBJECTIVE

To build and demonstrate a real end-to-end certificate lifecycle — issue, anchor, verify, revoke — in which the authenticity answer is derived from a public blockchain rather than from the issuing institution's database or from the PDF itself. The master spec is explicit that the goal "is not to create a mockup or a static demonstration" and requires "No fake blockchain transaction data in the final demo" (`CertiProof_Master_Build.md:11-29`, `128-157`). [IV]

1.8 SPECIFIC OBJECTIVES (DERIVED FROM WHAT IS ACTUALLY IMPLEMENTED)

1. Implement a single, specified, backward-compatible canonical metadata fingerprint protocol as the certificate identity. [IV]
2. Anchor that fingerprint, the CID, the issuer, and the timestamp in an immutable on-chain registry with revocation. [IV]
3. Keep document bytes in IPFS while keeping the credential (Pinata JWT) server-side only. [IV]
4. Provide three independent verification entry points that converge on one chain-authoritative result. [IV]
5. Make database metadata untrustworthy by construction: reconcile every displayed row against the chain and strip or drop anything that does not anchor. [IV]
6. Report transaction outcomes only from real mined receipts, with pending as a first-class non-success state. [IV]
7. Measure the pipeline (issuance, IPFS, verification) with reproducible harnesses and record the raw results. [IV] [EM]

1.9 CORE CONTRIBUTION OF THE PROJECT

The core contribution is a **layered trust architecture with an explicit authority boundary**, expressed in enforceable code rather than in prose:

- one canonical fingerprint function used identically at issuance, at database write-time checks, at Method A enrichment, and at Method C recomputation (`src/lib/certificate-fingerprint.ts` — the single source of truth);
- strict domain separation between the metadata fingerprint (registry key) and the document byte hash (file integrity only), never compared to each other;
- a database that cannot lie about a certificate because every row must re-hash to the on-chain anchor before its metadata is displayed, and because status is always overridden from the chain.

Secondary contribution: a reproducible experimental record of that design under real testnet conditions (IPFS 39/39 uploads, 36/36 byte-identical retrievals; verification 10/10 A, 10/10 B-consistency, 10/10 C; fingerprint consistency 23/23; a 123-test offline suite). [EM]

1.10 WHAT MAKES CERTIPROOF DIFFERENT FROM A NORMAL CERTIFICATE DATABASE

Property	Normal certificate database	CertiProof
Who can assert existence	The DB operator	A public contract any party can read (`getCertificate`) [IV]
Can rows be silently edited	Yes	Identity/status rows are reconciled against the chain; mismatching rows are dropped and metadata that does not re-hash is stripped (`reconcileSupabaseRowsWithChain` , `isSupabaseMetadataConsistentWithChain`) [IV]
Revocation	Mutable column	On-chain `status` 0→1 with an event, irreversible in code (`CertChainRegistry.sol:151-163`) [IV]
Document binding	A file path or blob	Content-addressed CID stored on-chain + a fingerprint that can be recomputed from the PDF's own printed fields (Method C) [IV]
Verifier needs an account	Usually	No wallet, no gas: read-only JSON-RPC provider (`metamask.ts:46-52`) [IV]
Failure honesty	Often conflated with success	Timeout ⇒ `pending` , never `confirmed` ; `blockNumber: 0` receipts rejected (`src/lib/blockchain/confirmation.ts`) [IV]

What it is NOT: it does not prove that the underlying academic claim is true, that the student earned the credential, or that the institution's real-world authorization is legitimate. `context_md/audit.md:228-230` states this limit explicitly and it must be preserved in the paper.

---

2. Research Problem

2.1 PROBLEM STATEMENT

Academic certificates are usually distributed as self-contained PDFs whose authenticity can only be checked by contacting the issuing institution. Because the PDF is not bound to any external, append-only record, a modified copy is indistinguishable from a genuine one, and verification does not scale beyond the institution's own willingness to respond. CertiProof addresses the narrower, technically tractable version of this problem: **how can a third party determine, without trusting the institution's database and without owning a wallet, whether a given certificate identity (a) exists, (b) was registered by an authorized issuer, (c) has not been revoked, and (d) still corresponds to the metadata printed on a presented document?** [IV]

2.2 EXISTING-SYSTEM LIMITATIONS (EVIDENCE-BASED)

Limitation	Evidence
Manual, institution-dependent verification	`CertiProof_Master_Build.md:47-61`
Tamper-vulnerable documents	Task 6 measured 2/4 PDF metadata tamper cases undetected under the pre-Task-7 extractor (`benchmarks/verification-results.json` → `reliability.tamperDetected = "2/4 (50%)"`) [EM]
Mutable centralized metadata	`context_md/2026-10-16_132041.md:30-37` — the detail page previously rendered Supabase rows as “Verified” with no chain cross-check
Database status can be flipped by any client holding the anon key	`supabase/rls-hardening.sql:32-47` documents that a hostile client can insert format-valid rows and update `status` , with the app-side reconciliation as the compensating control [EXT]
Public-RPC log retrieval is unreliable	`getLogs` from `fromBlock: 0` failed 0/10 in the verification benchmark; recorded as `blockchainRecordPass: false` on every Method A run [EM] [OL]
Verification latency grows with registry size	Method A `searchAndDecision` = mean 6019.64 ms of mean total 6022.47 ms (~99.9%) — the linear scan dominates (`benchmarks/verification-results.json`) [EM] [OL]

2.3 SECURITY PROBLEMS

Forgery of certificate content (recipient, course, date, ID); impersonation of an issuer (registering a certificate under a legitimate-looking identity); status fraud (claiming an active certificate is revoked, or a revoked one is active); metadata poisoning (displaying a fake recipient/status for a real certificate); wrong-network transactions that mine as no-ops and appear successful; and credential leakage from the client bundle. Each of these is analysed against real code in Section 18.

2.4 TRUST PROBLEMS

The verifier must not have to trust: the institution’s database (mutably controlled), the presentation layer (arbitrary HTML), the PDF’s visual appearance (trivially editable), or the QR image (can be re-pointed). CertiProof relocates trust to the contract’s `certificateHash` + `issuerAddress` + `status` , and treats everything else as untrusted input that must re-hash or be discarded. [IV]

2.5 CENTRALIZATION PROBLEMS

Supabase and the free public RPC are centralized points of failure/behavior; the contract itself has a single `superAdmin` with no ownership transfer (`CertChainRegistry.sol:16,46`) and issuer authorization is a single boolean mapping. These are real centralization residuals inside a decentralized anchoring design and must be disclosed, not hidden. [OL]

2.6 CERTIFICATE FORGERY / TAMPERING PROBLEMS

Two distinct forgery surfaces exist: **(a) forging a document that claims an existing identity** — defeated by Method C, which re-extracts printed fields and requires the recomputed fingerprint to equal the on-chain hash; and **(b) forging a new identity that does not exist on-chain** — defeated by Method A/B, which return `not_found` . What remains possible is forging a document for an identity that was never registered, which is necessarily `not_found` , and visually altering a certificate in ways the text extractor does not read (no OCR — see Section 14). [IV] [OL]

2.7 VERIFICATION PROBLEMS

Three are concrete and measured: (1) linear-scan latency, ~6 s mean per successful lookup and ~10–11 s for a miss; (2) `eth_getLogs` over a free RPC being capped (10,000-block range) so registration-event scans fail at ~50M blocks; (3) Method C’s dependence on the exact jsPDF text layout and on a Supabase metadata row existing (chain-only certificates yield `not_found` with a “metadata unavailable” marker rather than a full comparison). [EM] [OL]

2.8 AVAILABILITY / DEPENDENCY PROBLEMS

Public Amoy RPC throttling (recorded in `scripts/bench-verification.ts:307` — “Public RPCs throttle bursts”), Pinata availability/quota, Supabase availability, and browser-session-only state (pending-transaction registry lost on reload). RPC errors are handled asymmetrically by design: fail-open for read availability during reconciliation, fail-closed for writes (`src/lib/certificates.ts:369-385` vs `246-272`). [IV] [OL]

2.9 DATA INTEGRITY PROBLEMS

The historic integrity defect is documented precisely: issuance hashed canonical metadata while the forensic path hashed raw PDF bytes, so a legitimate upload could never match (`context_md/2026-10-14_204722.md:20-25`). This was measured live (“PDF byte-hash vs metadata fingerprint MISMATCH confirmed live”) and then fixed by establishing one canonical protocol with strict domain separation. [IV] [EM]

2.10 METADATA POISONING RISKS

A row in `certificates` can carry a fabricated recipient, course, status, or transaction hash while reusing a real `certificate_id`. Mitigations implemented: fingerprint gating of displayed metadata, row reconciliation that drops non-anchoring rows, status always overridden from chain, detail-page chain cross-check, and a status update constrained by both `certificate_id` and the chain-derived `certificate_hash`. See Section 17. [IV]

2.11 WHY CONVENTIONAL PDF CERTIFICATES ALONE ARE INSUFFICIENT

Because a PDF is a freely editable byte stream with no intrinsic binding to any authority: changing one printed character changes nothing about the file’s validity in the eyes of any viewer, and the file carries no authenticated statement about its own provenance. CertiProof’s answer is to hash a canonical form of the certificate’s *semantic* content and anchor that hash externally, then (Method C) rebuild the semantic content from a presented PDF and require re-derivation of the same hash. This is the technical core of the paper’s argument. [IV]

A. RESEARCH GAP

Supported gap (defensible from this repository): existing blockchain-credential work and deployed systems tend to (i) treat the stored record as the whole story, without an explicit authority hierarchy between an immutable chain record and a mutable metadata store, and (ii) verify uploaded documents against a hash of file bytes, or against database state, rather than against a canonical metadata fingerprint re-derived from the document itself. This project closes both: a chain-authoritative reconciliation boundary that survives a poisoned metadata store, and a document path that recomputes the registry fingerprint from extracted fields with strict hash-domain separation.

Requires literature verification before it is claimed as a gap: whether prior work already combines IPFS + SHA-256 metadata fingerprints + chain-vs-DB reconciliation in academic credentialing. The repository contains ten reference papers (see `context_md/2026-10-14_171533.md:45-60`) but contains **no completed related-work comparison table**. Marked **REQUIRES LITERATURE VERIFICATION**.

Explicitly not claimed: `context_md/2026-10-14_171533.md:79` records the honest finding that this project has **”NO algorithmic novelty.”** Do not claim a novel cryptographic algorithm.

B. RESEARCH QUESTIONS

Derived from what the implementation and experiments can actually answer:

- **RQ1.** Does anchoring a canonical SHA-256 metadata fingerprint on a public blockchain allow a wallet-free third party to distinguish authentic, revoked, and non-existent certificates without trusting the issuer’s database? (Answered by Methods A/B, 10/10 + 5/5 + 4/4 measurements.)
- **RQ2.** Can metadata that is recomputed from a presented PDF document be matched to an on-chain anchor, and does that detect metadata tampering? (Answered by Method C: 10/10 authentic `match`; tamper cases below.)
- **RQ3.** Does a chain-authoritative reconciliation layer prevent a malicious or corrupted metadata store from displaying false certificate information? (Answered by the 14 `supabase-integrity` tests plus the live 24/24 reconciliation.)

- **RQ4.** What are the measured latency and reliability characteristics of such a system on a public testnet and public IPFS pinning service? (Answered by the three benchmark JSONs.)

Not supported by evidence and therefore **not** posed: usability RQs, cost-vs-alternative RQs, mainnet scalability RQs.

C. RESEARCH OBJECTIVES

Matched 1:1 to RQ1–RQ4: implement the anchored fingerprint registry; implement document re-derivation and comparison; implement and evaluate the chain-authoritative reconciliation boundary; and measure end-to-end issuance, storage, and verification performance and reliability in a real testnet environment.

D. PROPOSED HYPOTHESIS

Not explicitly defined as a hypothesis in the repository. No file states a testable hypothesis with H0/H1 form. A hypothesis *consistent with* (not stated by) the work would be: *"Anchoring a canonical metadata fingerprint on-chain, combined with re-derivation of that fingerprint from a presented document, detects metadata tampering while preserving verification availability when the auxiliary metadata store is untrusted."* Mark **NOT EXPLICITLY DEFINED — proposed formulation requires author approval before use.**

---

3. Research Gap

Consolidated for direct use (see also Section 2.A and Section 26).

#	Gap statement	Status
G1	Verification systems commonly trust a mutable metadata store; there is no enforced authority boundary between the immutable record and the display layer. Demonstrated concretely here: the detail page once rendered unverified rows as "Verified" (`context_md/2026-10-16_132041.md:30-32`).	Implementation-closed; **literature comparison required**
G2	Document verification commonly hashes file bytes and compares against a metadata anchor, which can never match (measured live, `2026-10-14_204722.md:20-25`).	Implementation-closed; **literature comparison required**
G3	Recipient/course tampering in PDFs was undetectable when field extraction was placeholder-based; measured 2/4 detection pre-fix.	Closed by Task 7; **literature comparison required**
G4	Transaction "success" was previously reported from a fabricated `blockNumber: 0` receipt when the wait timed out; measured 5/10 bench runs hit the timeout and were assumed successful while mining 37–166 s later.	Closed by Task 4; **likely novel as an engineering finding — requires literature verification**
G5	Wrong-chain signing can mine as a no-op at an address without code and be classified as success.	Closed by Task 8 guard; **requires literature verification**
G6	Scalability of linear on-chain registry scan is unaddressed.	**OPEN — future work**

---

4. Objectives

4.1 MAIN OBJECTIVE

Deliver a working, demonstrable, non-simulated certificate lifecycle in which authenticity is determined from a public chain record. [IV]

4.2 SPECIFIC OBJECTIVES (IMPLEMENTATION-MAPPED)

#	Objective	Implementation locus	Status
O1	Single canonical fingerprint protocol, backward compatible with existing records	`src/lib/certificate-fingerprint.ts`; proven by `scripts/check-fingerprint-compat.ts` (24/24 stored + 24/24 chain MATCH)	Achieved [IV] [EM]
O2	On-chain registry with issuer authorization and revocation	`contracts/CertChainRegistry.sol`, deployed at `0x0992…DF50`	Achieved [IV]
O3	Server-side credential handling for IPFS	`src/lib/ipfs.ts` + `src/lib/server/pinata.ts` (server-only import barrier)	Achieved [IV]
O4	Wallet-free verification via three methods	`verifyCertificate`, `buildQrPayload`/`extractHashFromQrPayload`, `analyzeDocument`	Achieved [IV]
O5	Database cannot present unanchored data as verified	`reconcileSupabaseRowsWithChain`, `fetchVerifiedCertificateFromSupabase`, metadata fingerprint gate	Achieved (app-side) [IV]; **RLS [EXT]**
O6	Honest transaction outcome reporting	`src/lib/blockchain/confirmation.ts` + 29 tests	Achieved [IV]
O7	Reproducible measurement of the pipeline	`scripts/bench-*.ts` → `benchmarks/*.json`	Achieved, with sample-size caveats [EM]
O8	Scale beyond linear scan	—	**Not achieved [FW]**
O9	Production deployment / auth hardening	—	**Not achieved [FW] [EXT]**

4.3 DELIVERABLES ACTUALLY PRESENT IN THE REPOSITORY

`contracts/CertChainRegistry.sol` (179 lines), `src/lib/` service layer, `src/routes/` (12 routes), `supabase/rls-hardening.sql` (proposal), `test/` (7 files, 123 tests), `scripts/` (14 scripts: deploy, add-issuer, check-contract, eval-metrics, 3 benchmarks, 5 live checks, fingerprint-compat), `benchmarks/` (3 JSONs), `context_md/` (20 session files + audit). [IV]

4.4 DELIVERABLES PLANNED BUT ABSENT

- `test/CertChainRegistry.test.ts` — planned in `BUILD_PLAN.md:68`, **does not exist**; there is **no smart-contract test suite**. [OL]
- `getTransactionHistory()` — listed in `BUILD_PLAN.md:61`, **not implemented** in the contract (11 functions, no such function). [OL]
- `hardhat.config.ts` / `scripts/deploy.ts` (TypeScript) as planned — actual files are `hardhat.config.cjs` / `scripts/deploy.cjs`. [IV]
- CI (`.github/`) — absent. [OL]
- A `typecheck` npm script — absent; typecheck is run manually as `bunx tsc --noEmit`. [OL]

5. System Architecture

5.1 COMPONENT INVENTORY

#	Component	Purpose	Input	Output
C1	Frontend React/TS app (`src/`)	UI + orchestration	User input	Renders state
C2	TanStack Start server fn (`src/lib/ipfs.ts`)	Boundary for secret-bearing work	`FormData`	`{cid,url}` or safe error
C3	`src/lib/server/pinata.ts`	Validated IPFS pinning	`FormData`	CID
C4	Smart contract `CertChainRegistry`	Immutable registry	`registerCertificate` / `revokeCertificate`	State + events
C5	Read-only JSON-RPC provider (`getReadOnlyProvider`)	Wallet-free reads	RPC calls	Chain data
C6	MetaMask / signer	Transaction signing	User approval	tx hash
C7	Fingerprint module (`certificate-fingerprint.ts`)	Canonical identity derivation	4 metadata fields	`0x` +64 hex
C8	SHA-256 helper (`crypto.ts`)	WebCrypto digest + bytes32 helpers	string/bytes	hex
C9	PDF generator (`pdf/generate-certificate.ts`)	Renders the certificate	Certificate data	PDF `Blob`
C10	PDF extractor (`pdf/extract-certificate-metadata.ts`)	Re-derives fields from a PDF	PDF bytes	`{id,name,course,date,complete`
C11	Supabase (`certificates` table)	Display/index metadata	INSERT/SELECT/UPDATE	Rows
C12	Pinata API + gateway	IPFS pinning & retrieval	file bytes	CID / bytes
C13	Confirmation state machine	Receipt classification	tx hash	`confirmed|failed|pending`
C14	Pending transaction registry	Preserves unconfirmed work	tx + payload	in-memory records
C15	Verification UI (`verify`, `verify_.$hash`,	Presents results	hash / file / id	Outcome + checks

#	Component	Purpose	Input	Output
	`certificate.$id`)			
C16	Dashboard + bulk issuance	Issuer operations	CSV / forms	Batch results
C17	QR generator (`qrcode`)	Encodes verification URL	fingerprint	PNG data URL
C18	Wallet hook (`useWallet.tsx`)	Connection + chain state	EIP-1193 events	`WalletState`

5.2 STACK (RESOLVED FROM `PACKAGE.JSON` / `BUN.LOCK`)

React 19.2.8, TypeScript 5.9.3, TanStack Start 1.168.32 / Router 1.170.18, Vite 8.2.1, ethers 6.17.0, Hardhat 2.29.1 (Solidity 0.8.24, optimizer runs 200), `@supabase/supabase-js` 2.112.4, jsPDF 4.2.1, `qrcode` 1.5.4, Tailwind CSS v4, shadcn/ui + Radix, React Hook Form + Zod, Recharts, Bun (runtime, package manager, test runner). [IV]

5.3 TRUST-BOUNDARY DIAGRAM (TEXTUAL)

```
UNTRUSTED ZONE                          TRUSTED / AUTHORITATIVE ZONE
--------------                          ----------------------------
Browser DOM, QR image, uploaded PDF  →  Parser/validator (C10, C3)
Supabase rows (anon key, any client) →  Reconciler vs chain (C13/C4)
Displayed metadata                   →  fingerprint gate (C7 → C4 hash)
Wallet provider (user-controlled)    →  wrong-network guard → chain (C4)
Public RPC availability              →  receipt validation rules (C13)
IPFS retrieved bytes                 →  SHA-256 byte compare (benchmark only)
```

Authority order for any certificate fact: **contract state > event/receipt evidence > recomputed fingerprint > database row > rendered text.** [IV]

5.4 COMPONENT NOTES WORTH CARRYING INTO THE PAPER

- The ABI used by the frontend is an **inline human-readable array** in `src/lib/blockchain/contracts.ts:1-22`, not imported from the Hardhat artifact; the artifact is consumed only by `scripts/deploy.cjs`. The two were checked to match. [IV]
- `src/lib/certiproof.ts` (289 lines) is the original **mock service layer and is dead code** — no module imports it (`context_md/2026-10-15_230829.md:96`). It is also the only place in the codebase where `documentByteHash` is conflated with `cert.hash` (lines 230, 250) — a conflation that exists **only in unimported dead code**. [OL]
- `src/lib/blockchain/verify.ts`'s `computeStatus` / `verifyOnChain` / `compareHash` / `certificateExists` are largely unreferenced; only `checkIssuerAuth` is live (used by `login.tsx:48`). [OL]
- `src/lib/demo-data.ts` still holds a **different contract address** (`0x9C41…E30`) from the deployed one (`0x0992…DF50`) — see Section 35. [OL]

---

6. End-to-End Issuance

The real path is `src/lib/certificates.ts` → `issueCertificate(...)`, called by `src/routes/issue.tsx` (three-step wizard: `STEPS = ["Recipient details", "Certificate details", "Review & issue"]`, `issue.tsx:58`) and by `src/routes/issue_.bulk.tsx` (CSV batch).

6.1 EXACT STAGE SEQUENCE

Instrumented stage names come from `src/lib/issuance-timing.ts` (`fingerprint`, `pdf`, `ipfs`, `blockchainPreparation`, `blockchainSubmission`, `blockchainConfirmation`, `supabasePersistence`).

#	Stage	What happens	Data generated	Where it goes	Crypto
0	**Wrong-network guard**	`isConnectedToAmoy()` must return true, else typed `wrong_network` `IssuanceError` is thrown and a failure record is written with `transactionSubmitted=false`	none	failure record	none
1	`fingerprint`	`generateCertificateFingerprint({certificateId, recipientName, course, issueDate})`	canonical string; `0x` +64 hex	in-memory `Certificate.hash`	SHA-2 (WebC over U
2	`pdf`	`generateCertificateBlob(...)` via jsPDF (landscape A4, helvetica, static labels)	`Blob` → `File`	memory	none
3	`ipfs`	`uploadToIPFS(file)` → TanStack server fn → `uploadCertificateFile` → Pinata `pinFileToIPFS`	`cid` , `url`	Pinata (bytes), CID returned to client	none (transp TLS)
4	`blockchainPreparation`	`getSigner()` + `getContract(signer)`	signer/contract handles	—	none
5	`blockchainSubmission`	`submitTransactionAndConfirm` → `registerCertificate(certId, bytes32 hash, cid, certId)` with `maxPriorityFeePerGas` 30 gwei / `maxFeePerGas` 50 gwei, no explicit `gasLimit`	tx hash	broadcast to Amoy	ECDS signatu wallet
6	`blockchainConfirmation`	`waitForTransactionConfirmation` — `tx.wait()` raced against `CONFIRMATION_TIMEOUT_MS = 60_000` , then one final receipt check	`confirmed` | `failed` | `pending` + receipt	—	none
7	Certificate construction	`buildIssuedCertificate` from the **real** `receipt.hash` and `receipt.blockNumber`	`Certificate` object	in-memory store (`addToStore`)	none
8	`supabasePersistence`	`saveCertificateToSupabase(cert, cid)` — **non-blocking** in the app; awaited by the benchmark harness only	INSERT row	Supabase `certificates`	none
9	QR (post-issuance)	`CertificatePreview` renders `https://certiproof.app/verify/<hash>`	PNG data URL	UI only	none

QR generation is deliberately **not** part of the issuance timing path (`context_md/2026-10-14_231247.md:15` , and `runs[0].qrGenerationMs = null` in the benchmark JSON); a separate microbenchmark measured QR generation at 136.43 ms (current JSON) / 94.45 ms (Task 3 session). [EM]

CONFIDENTIAL

6.2 WHAT IS STORED WHERE

Store	Contents	Not stored
On-chain (`CertChainRegistry`)	`certificateId` (mapping key + `certificateIds` array), `certificateHash` (bytes32), `issuerAddress`, `ipfsCID`, `certificateNumber` (the app passes `certId` again), `issueTimestamp = block.timestamp`, `status`	recipient name, recipient email, course, type, organization, human `issueDate`, description, PDF bytes, tx hash, block number
IPFS (Pinata)	the PDF bytes	metadata, keys
Supabase `certificates`	`certificate_id, recipient_name, recipient_email, course, certificate_type, certificate_hash, transaction_hash, issuer_wallet, ipfs_cid, issue_date, status` (+ `id`, `created_at`)	chain state authority
In-memory only	pending-transaction records, dashboard store	lost on reload

Evidence that chain-fetch paths explicitly blank metadata: `src/lib/certificates.ts:462-465`, `521-524`, `777-780` all set `recipientName: ""`, `course: ""`, `type: "Completion"`. [IV]

6.3 TRANSACTION, CONFIRMATION, RECEIPT

- On `confirmed`: certificate built from `receipt.hash` / `receipt.blockNumber`; `receipt.status === 1` required. Supabase insert happens **only after** this point (`certificates.ts:1180-1183`). [IV]
- On `pending` (timeout with no receipt): `IssuanceError { type: "confirmation_pending", transactionHash }` is thrown; `recordPendingTransaction` stores `txHash, certificateId, fingerprint, ipfsCid, issuerWallet, input, submittedAt`; **no certificate object is returned, no store entry, no Supabase row** (`certificates.ts:1105-1139`). [IV]
- On `failed`: typed `transaction_failed`; the batch stops at the first failure and the UI shows “Batch stopped” (`issue_.bulk.tsx`). [IV]
- A later user-triggered one-shot re-check (`checkPendingTransaction`) may promote a pending record: only when a real receipt with `status === 1` and `blockNumber > 0` is found does the certificate get built and persisted (`certificates.ts:1340-1379`). [IV]

6.4 WRONG NETWORK BEHAVIOUR

The guard runs **before** fingerprint/PDF/IPFS/signing. On a non-Amoy chain the operation throws `wrong_network` with message *”Wrong network. Please switch MetaMask to Polygon Amoy before issuing.”*, records `transactionSubmitted=false`, `transactionHash=null`, and nothing else happens. Verified live with a shim reporting `eth_chainId 0x1`: only `eth_chainId` was ever observed, zero funds spent (`scripts/wrong-network-check.ts`; `context_md/2026-10-16_152855.md:26-29`). [IV] [EM]

Rationale recorded in code (`certificates.ts:986-990`): a call to an address without code still mines with receipt status 1, which the confirmation machine would classify as success — producing a false “anchored” result plus wasted gas and a wasted IPFS pin. [IV]

6.5 FAILURE MODES SUMMARY

Condition	Outcome	User-visible
Wrong chain	`wrong_network`	typed error, no tx, no IPFS spend
Pinata failure	`ipfs` stage failure	generic safe message (see Section 9)
Submission throws / no hash	`failed` / `submission_error`, `txHash: null`	failed panel
Revert (`status 0`)	`failed` / `mined_reverted`	failed panel
Timeout, no receipt	`pending` / `timeout_no_receipt`	amber pending panel + PolygonScan link + “Check confirmation status”
RPC error during re-check	`pending` (never success)	still pending
Supabase insert fails	non-blocking; console error; chain record unaffected	none

7. Cryptographic Fingerprint Protocol

Single source of truth: `src/lib/certificate-fingerprint.ts` (76 lines). The header comment is normative and is reproduced in substance below.

7.1 INPUTS, ORDERING, CANONICALIZATION

```
fields (fixed order):  certificateId, recipientName, course, issueDate
separator:             "|"
per-field:             trim() on certificateId, recipientName, course
                       normalizeIssueDate(issueDate)
join:                  [certificateId, recipientName, course, issueDate].join("|")
encode:                UTF-8 (TextEncoder)
hash:                  SHA-256 via crypto.subtle.digest
output:                "0x" + lowercase hex, 64 chars  →  /^0x[0-9a-f]{64}$/
```

`normalizeIssueDate(value)`:

1. `trim()`;
2. if it matches `^(\d{4})-(\d{1,2})-(\d{1,2})(?:[T ].*)?$`, keep the date part;
3. zero-pad month and day to width 2 → `YYYY-MM-DD`;
4. otherwise return the trimmed value **verbatim** (e.g. `18/05/2026`, `May 18, 2026` pass through unchanged).

Rules that matter and are test-pinned: leading/trailing whitespace is removed; **internal whitespace is significant**; object key insertion order does not matter; `2026-5-18` and `2026-05-18` produce the same fingerprint; Supabase `TIMESTAMPTZ` values (`2026-05-18T00:00:00+00:00`) reduce to the date component so they round-trip to the same fingerprint as the plain date; Unicode names are hashed as UTF-8. [IV]

7.2 PSEUDOCODE (DERIVED LITERALLY FROM THE IMPLEMENTATION)

```
FUNCTION canonicalizeCertificateMetadata(m):
    RETURN join("|",
        trim(m.certificateId),
        trim(m.recipientName),
        trim(m.course),
        normalizeIssueDate(m.issueDate))

FUNCTION generateCertificateFingerprint(m):
    canonical := canonicalizeCertificateMetadata(m)
    hex       := hexLower(SHA256(UTF8Bytes(canonical)))      // crypto.subtle.digest
    RETURN "0x" + hex
```

7.3 PROTOCOL VECTOR (FROM TESTS)

Canonical base string: `CERT-2026-100001|Asha Kumar|B.Tech Computer Science and Engineering|2026-05-18`, asserted against an independent WebCrypto oracle, plus a legacy-equivalence assertion proving byte-identity with the original `id|name|course|date` scheme (`test/certificate-fingerprint.test.ts:21-22, 98-108`). [IV]

7.4 STORAGE AND RECOMPUTATION

Location	How
Chain `bytes32 certificateHash`	`hexToBytes32(hash)` then `registerCertificate(certId, hashBytes32, cid, certId, …)` (`certificates.ts:1066, 1083-1092`)
Supabase `certificate_hash`	`cert.hash.toLowerCase()` (`certificates.ts:233`)
QR URL	`https://certiproof.app/verify/<hash>`
Pending record	field `fingerprint`

Recomputed with the **same function** at: issuance (`certificates.ts:1019-1026`); Method A metadata gate (`349-355`); Method C registry check (`1495-1500`); Method C document check (`1530-1537`); detail-page verification (`414`); audit script `scripts/check-fingerprint-compat.ts:64-69`. [IV]

7.5 THE THREE HASHES — DEFINITIONS AND TRUST RELATIONSHIP

Name	Exact definition	Computed where	Stored where	Ever compared to
metadataFingerprint (`Certificate.hash`)	SHA-256 of the canonical 4-field string	issuance + all verification recomputations	chain bytes32, Supabase `certificate_hash`, QR URL	on-chain hash; Supa `certificate_hash` recomputed peers
documentByteHash	SHA-256 of the *raw uploaded file bytes*	Method C stage 0 (`certificates.ts:1407-1408`)	returned in `ForensicResult.documentByteHash` only	**never compared to anything** — display `"n/a — not the registry key"` (`certificates.ts 1464`)
on-chain hash	`result[0]` of `getCertificate(id)` — the registered bytes32	read from the contract	contract `certificates` mapping	the reference value Method A input, both Method C fingerprin and the metadata g

CONFIDENTIAL

Are they ever conflated? In production code: **no.** Explicit statements of separation exist at `certificate-fingerprint.ts:15-17`, `types.ts:70-75`, `certificates.ts:1393-1399`, `extract-certificate-metadata.ts:37-39`, and `bench-ipfs.ts:675-676`. A test asserts `documentByteHash === SHA-256(file bytes)` independently of any registry value (`test/verification.test.ts:83-90`).

The **only** place the two are conflated is `src/lib/certiproof.ts` lines 230 and 250 (`documentByteHash: mockHash(fileName)` and `documentByteHash: cert.hash`), which is **dead code imported by nothing**. Reported here so the paper never cites it as behaviour. [OL]

Trust relationship: the on-chain hash is the anchor; the metadata fingerprint is *derived*, so equality with the chain value is the authenticity test; the document byte hash is an independent integrity reference with **no** authoritative counterpart anywhere in the system.

7.6 BEHAVIOUR IF ONE FIELD CHANGES

Any single-character change to any of the four fields (including internal whitespace changes, since internal whitespace is significant) changes the canonical string and therefore the SHA-256, so the recomputed fingerprint no longer equals the on-chain anchor. Covered by dedicated tests for recipient, course, date, certificate ID, single-character mutation, and whitespace-equivalence. [IV]

7.7 KNOWN PROTOCOL PROPERTIES / CAVEATS

- No explicit null handling inside the protocol — the interface types all four fields non-nullable, and callers gate nulls upstream. [OL]
- Date normalisation is *not* a full date parser: non-ISO formats are hashed verbatim, so `18/05/2026` and `2026-05-18` are different identities. Documented by the test `non-ISO dates are kept verbatim (trimmed)`. [OL]
- No protocol version field exists; changing the canonical rules would break all existing records. Backward compatibility is therefore a hard constraint. [OL]

---

8. Smart Contract

File: `contracts/CertChainRegistry.sol` — the only `.sol` file in the repository, 179 lines.

8.1 IDENTIFICATION

Property	Value	Evidence
Contract name	`CertChainRegistry`	line 4
Pragma	`pragma solidity ^0.8.24;`	line 2
License	`// SPDX-License-Identifier: MIT`	line 1
Compiler (configured)	0.8.24, optimizer enabled, runs 200	`hardhat.config.cjs:6-11`
Inheritance / imports	**none** (no OpenZeppelin, zero `import` statements)	whole file
Network	Polygon Amoy testnet	multiple
Chain ID	**80002** (`0x13882`)	`deployment.json:4` , `hardhat.config.cjs:20` , `contracts.ts:25-26`
Contract address	**`0x09921D25cB513793e17082Cce0fB4F520f0CDF50`**	`deployment.json:2` (conflicting value in `src/lib/demo-data.ts:7` — see Section 35)
Deployer / super admin	`0x3e72b748a543DEc762a297F7304A04F980d7bC01`	`deployment.json:5`
Deployment timestamp	`2026-08-31T06:04:59.550Z`	`deployment.json:7`
Explorer	`https://amoy.polygonscan.com`	`deployment.json:6`
Constructor args	**none**	artifact ABI `inputs: []`

8.2 STRUCT

```
struct Certificate {
    bytes32 certificateHash;
    address issuerAddress;
    string ipfsCID;
    string certificateNumber;
    uint256 issueTimestamp;
    uint256 status; // 0 = Active, 1 = Revoked
}
```

8.3 STATE, MAPPINGS, ARRAYS

Declaration	Line	Notes
`address public superAdmin`	16	set once in constructor; **no transfer/renounce function exists**
`mapping(string => Certificate) public certificates`	17	id → record; auto-getter
`mapping(address => bool) public authorizedIssuers`	18	wallet → authorized
`address[] public issuerList`	19	append-only; **never pruned on removal**
`string[] public certificateIds`	20	append-only enumeration index

8.4 EVENTS

```
event IssuerAdded(address indexed issuer, uint256 timestamp);
event IssuerRemoved(address indexed issuer, uint256 timestamp);
event CertificateRegistered(string indexed certificateId, bytes32 indexed certificateHash,
                            address indexed issuer, uint256 timestamp);
event CertificateRevoked(string indexed certificateId, uint256 timestamp);
```

`CertificateRegistered` has three indexed parameters; `timestamp` is not indexed. [IV]

8.5 MODIFIERS AND FUNCTIONS

Modifiers: `onlySuperAdmin` (`require(msg.sender == superAdmin, "Only super admin")`), `onlyAuthorizedIssuer` (`require(authorizedIssuers[msg.sender], "Not authorized issuer")`).

Function	Visibility	Modifier	Revert strings	Emits
`constructor()`	—	—	—	— (sets superAdmin, self-authorizes, pushes to `issuerList`)
`addIssuer(address)`	external	onlySuperAdmin	`"Invalid address"`, `"Already authorized"`	`IssuerAdded`
`removeIssuer(address)`	external	onlySuperAdmin	`"Not authorized"`	`IssuerRemoved`
`isAuthorizedIssuer(address)`	external view	—	—	—
`getIssuerCount()`	external view	—	—	— (O(n) scan counting mapping-true entries)
`registerCertificate(string, bytes32, string, string)`	external	onlyAuthorizedIssuer	`"Certificate already exists"`, `"Hash cannot be zero"`	`CertificateRegistered`
`getCertificate(string)` → 6-tuple	external view	—	`"Certificate not found"`	—
`certificateExists(string)`	external view	—	—	— (`bytes(ipfsCID).length != 0`)
`revokeCertificate(string)`	external	**onlySuperAdmin**	`"Certificate not found"`, `"Already revoked"`	`CertificateRevoked`
`getCertificateStatus(string)`	external view	—	`"Certificate not found"`	—
`getCertificateCount()`	external view	—	—	—

8.6 STATE TRANSITIONS

- **Registration:** requires no existing entry (checked by empty `ipfsCID`) and a non-zero hash; writes the struct with `issuerAddress = msg.sender`, `issueTimestamp = block.timestamp`, `status = 0`; appends the id; emits `CertificateRegistered`.
- **Revocation:** `status` 0 → 1 only, guarded by `"Already revoked"`; emits `CertificateRevoked`. **Irreversible in code** — no function writes `status = 0` after registration, so revocation cannot be undone without redeploying. [IV]

8.7 WHAT IS AND IS NOT PERMANENTLY ANCHORED

Anchored: certificate id, fingerprint bytes32, issuer address, IPFS CID, certificate number, issue timestamp, status, plus the event log (id, hash, issuer, timestamp). **Not anchored:** all human-readable metadata (recipient, course, email, type, human date), the PDF bytes, the transaction hash / block number (recoverable only from logs or receipt), and any presentation-layer data. [IV]

8.8 WHY THE BLOCKCHAIN IS AUTHORITATIVE FOR EACH FACT

Fact	Why authoritative	Code
Certificate existence	written only by `registerCertificate`; `getCertificate` reverts for unknown ids	`CertChainRegistry.sol:87-105, 130-133`
Certificate hash/fingerprint	stored bytes32, immutable, never rewritten	`81-105`
Issuer	`issuerAddress = msg.sender` at registration; only authorized wallets may register	`98`, `39-42`
Revocation status	`status` written only by `revokeCertificate`, event emitted	`151-163`
Issue timestamp	`block.timestamp` assigned by the chain	`101`
Transaction/block evidence	receipt and `CertificateRegistered` logs, verified for `status === 1` and matching contract/topic	`resolveRegistrationTx`, `certificates.ts:106-161`

Consequently, all application reads are implemented so that status comes from `result[5]` (the chain tuple), never from Supabase (`certificates.ts:737, 784`). [IV]

8.9 GAS

No gas figure is stored in any benchmark JSON. The only recorded gas numbers in the repository are prose in session notes: registration mean **241,189** gas over n=16 (range 240,060–257,964), revocation mean **60,932** over n=4, **≈0.007236 POL** at 30 gwei — recorded `context_md/2026-10-14_171533.md:32-34` against a 16-certificate registry era. `scripts/eval-metrics.ts:113-115` can recompute these but prints to console only; results are not persisted. Mark **[EM] but stale and non-persisted — must be re-measured before publication.**

8.10 SECURITY PROPERTIES AND LIMITATIONS VISIBLE IN CODE

Properties: no external calls, no `payable` functions, no `delegatecall`, no reentrancy surface; state written after all `require`s; every state transition emits an event; access control enforced at the contract (not the UI) layer. [IV]

Limitations: single immutable `superAdmin` with no transfer/renounce and no pause; no upgradability; existence keyed on `bytes(ipfsCID).length` rather than an explicit flag, so a registration with an empty CID would be written yet invisible to `getCertificate`; duplicate-ID protection depends on that same string check; unbounded `certificateIds` enumeration with no pagination; `status` is `uint256` rather than an `enum`; issuer authorization is checked only at registration time (revoking an issuer does not affect certificates they already registered); `issuerList` grows forever. [OL]

No automated smart-contract tests exist (`test/CertChainRegistry.test.ts` was planned in `BUILD_PLAN.md:68` but is absent). [OL] [EXT-adjacent]

---

9. IPFS / Pinata Architecture

9.1 FLOW

```
Browser  uploadToIPFS(file)                      src/lib/ipfs.ts
   ↓  FormData { file }
TanStack Start createServerFn({method:"POST"})   ipfs.ts:11-16
   ↓  handler dynamically imports ./server/pinata
Server  uploadCertificateFile(form)              src/lib/server/pinata.ts
   ↓  validateUploadForm(form)      → reject before any network call
   ↓  resolvePinataJwt()            → process.env.PINATA_JWT (server only)
   ↓  pinFileToIpfs(jwt, file)      → POST https://api.pinata.cloud/pinning/pinFileToIPFS
   ↓                                  header Authorization: Bearer <jwt>
   ↓                                  body pinataMetadata { name, keyvalues:{type:"certificate"} }
   ↓  reads payload.IpfsHash
returns { ok:true, cid, url:"https://gateway.pinata.cloud/ipfs/<cid>" }
```

The returned `url` is **never used by application code** — only `cid` is consumed (on-chain + Supabase). [IV]

9.2 VALIDATION RULES (EXACT)

Rule	Exact value / message
Size limit	`export const MAX_UPLOAD_BYTES = 10 * 1024 * 1024` → *"File exceeds the maximum upload size of 10 MB."*
Allowed MIME types	`application/pdf` → `.pdf`; `image/png` → `.png`; `image/jpeg` → `.jpg`, `.jpeg`
Unknown/missing MIME	*"Unsupported file type. Allowed types: PDF, PNG, JPEG."*
Extension vs MIME mismatch	checked **only if the filename contains a dot**; message *"File extension does not match its content type ()."*
No file extension at all	**accepted** (locked in by test `accepts a valid PDF without a file extension (current behavior locked in)`)
Empty file (`size <= 0`)	*"The uploaded file is empty."*
Not `FormData`	*"Invalid upload request."*
Missing `file` entry	*"No file was provided in the upload request."*
String entry	*"Invalid file upload."*

Validation runs **before** Pinata is contacted (test asserts `fetchCalled === false` for invalid input). [IV]

9.3 CREDENTIAL HANDLING AND THE SERVER/CLIENT BOUNDARY

- `PINATA_JWT` is read from `process.env` inside `src/lib/server/pinata.ts` only; `.env.example` documents it as `# Pinata IPFS (server-side only — never VITE_ prefixed)`.
- `src/lib/server/pinata.ts:1` imports `@tanstack/react-start/server-only`, whose import-protection plugin raises a violation if a client bundle pulls it in.
- No `VITE_PINATA*` variable exists anywhere in `src` (0 matches).
- Escaping path is a generic client message; tests assert the JWT and upstream error bodies never appear in returned messages.
- Task 8/10 secret scans of the built client bundle (`.output/public/assets/*.js`) found no `PINATA_JWT`, `DEPLOYER_PRIVATE_KEY`, `service_role`, `eyJhbGciOi`, or `PRIVATE_KEY`; `benchmarks/verification-results.json` → `security.findings: []`.

Historical defect, now closed: the original implementation read `import.meta.env.VITE_PINATA_JWT` and sent the bearer token from the browser — `context_md/audit.md:102-112` (C-05). Fixed in Task 2 (`context_md/2026-10-14_222146.md`). **Residual recommendation recorded in the audit: if the token was ever in a deployed bundle it should be rotated; the repository does**

not record whether rotation happened. [OL] [EXT]

9.4 ERROR HANDLING (COMPLETE SET)

Condition	Client-facing message	Server log
Network failure	`IPFS upload failed due to a network error. Please try again.`	`network error contacting Pinata: <msg>`
HTTP !ok (any status)	`IPFS upload failed. Please try again later.`	`Pinata rejected the request (status <n>)`
Non-JSON body	`IPFS upload failed. Please try again later.`	`Pinata returned an unreadable response.`
Missing/empty `IpfsHash`	`IPFS upload failed. Please try again later.`	`Pinata response did not contain a CID.`
Missing JWT	`IPFS upload is not configured on the server. Please contact the administrator.`	`PINATA_JWT is not configured on the server.`
Unexpected throw	`IPFS upload failed. Please try again.`	`unexpected error: …`

Status codes go only to the server log, never to the client. [IV]

9.5 RETRIEVAL, INTEGRITY, DEDUPLICATION

- **Application code performs no retrieval.** No gateway fetch, no `retrieveFromIPFS`, no CID re-hash exists in `src/`. [OL]
- **Benchmark does**, with an explicit protocol: *"Integrity is proven by SHA-256(original bytes) === SHA-256(retrieved bytes); an HTTP 200 alone is NEVER treated as success"* (`scripts/bench-ipfs.ts:8-9`).
- **Deduplication** is not an application feature; it is an observed property of content addressing (Phase C).

9.6 BENCHMARK EVIDENCE — `BENCHMARKS/IPFS-RESULTS.JSON`

Generated `2026-10-15T16:18:02.772Z`; gateway `https://gateway.pinata.cloud/ipfs`; `runsPerSize: 5`; 7 sizes (10KB…10MB); 36 timed cycles + 3 dedup uploads.

Reliability (as stored):

Metric	Value
Upload success	`39/39 (100%)`
Retrieval, first attempt	`36/36 (100%)`
Retrieval with retries (max 3, 2500 ms apart)	`36/36 (100%)` — no retry was ever needed
Byte-identical integrity	`36/36 (100%)`
Delayed recheck of the certificate PDF	performed after `652739.15` ms; retrieval `6535.42` ms; HTTP 200; `integrityOk: true`
Dedup	`sameFileCidEqual: true`, `differentFileCidEqual: false`
Error probes	`4/4` behaved as expected, `secretLeaked: false` for all
Unpin cleanup	attempted 39, succeeded 37 (see caveat below)

Upload latency (ms), n=5 per size:

Size	min	median	mean	max	stdDev
10KB	1070.15	1162.64	1154.14	1259.23	73.60
100KB	1161.19	1305.61	1400.51	1970.55	325.82
500KB	2124.59	2347.09	2540.10	3216.72	452.15
1MB	3048.52	4147.48	4106.02	5214.04	828.88
2MB	5816.96	7303.86	7517.13	9208.48	1309.32
5MB	14962.96	17787.12	20527.66	35496.26	8520.59
10MB	19417.93	27612.86	30805.56	54534.18	13722.25
Overall (n=36)	**1070.15**	**3881.22**	**9508.25**	**54534.18**	**12037.31**

Retrieval latency (ms), n=5 per size:

Size	min	median	mean	max	stdDev
10KB	4932.24	7035.29	6598.31	8002.00	1502.33
100KB	4713.61	4917.61	5445.85	6865.40	934.36
500KB	4593.96	6360.82	6211.91	7615.10	1307.88
1MB	5674.64	6810.77	7106.93	8858.69	1367.21
2MB	7618.90	8306.94	8354.96	9544.57	743.89
5MB	6272.89	8847.30	8067.88	9517.33	1641.12
10MB	6503.62	8858.82	8703.52	11009.05	1924.88
Overall (n=36)	**4593.96**	**7045.28**	**7181.63**	**11009.05**	**1687.95**

Certificate PDF (4795 B): upload `2041.45` ms, retrieval `6091.88` ms, integrity true.

Error probes (exact `resultSummary`): missing credential → `IPFS upload is not configured on the server. Please contact the administrator.` ; invalid credential → `IPFS upload failed. Please try again later.` ; malformed request → `Invalid upload request.` ; unknown CID → `httpStatus=400 bodyNonEmpty=true durationMs=6436.77` .

Honest interpretation limits: these are single-run-per-cycle observations against one vendor gateway on one day with no warm-up phase; variance at 5–10 MB is very large (stdDev 8.5 s and 13.7 s); retrieval has a ~5–9 s floor regardless of size, indicating gateway/latency dominance rather than throughput. They must not be generalised into “IPFS is 100% reliable.” [EM] [OL]

Unpin caveat: the stored `37/39` counts only dedup and size-sweep unpins; the certificate-PDF unpin outcome is console-only by design, and the second dedup unpin intentionally returns HTTP 400 “already unpinned”. Two “failures” ≠ two cleanup failures (DERIVED interpretation from `bench-ipfs.ts:729-735`). [OL]

---

10. Supabase Architecture

10.1 CLIENT AND SCHEMA

`src/lib/supabase.ts` creates a browser client from `VITE_SUPABASE_URL` + `VITE_SUPABASE_ANON_KEY` (both `VITE_` -prefixed and therefore public by design). One table, **`certificates`** , 12 columns:

`id` (number, optional), `certificate_id` , `recipient_name` , `recipient_email` , `course` , `certificate_type` , `certificate_hash` , `transaction_hash` , `issuer_wallet` , `ipfs_cid` , `issue_date` , `status` , `created_at` (optional).

No RPCs, no other tables, no `.delete()` call anywhere in `src/`. [IV]

10.2 OPERATIONS

Operation	Function	Query
SELECT all	`fetchAllCertificatesFromSupabase`	`.from("certificates").select("*").order("created_at", {ascending:false})`
SELECT by id	`fetchCertificateFromSupabase`	`.eq("certificate_id", certId).single()`
SELECT by hash	`fetchCertificateByHashFromSupabase`	`.eq("certificate_hash", hash.toLowerCase()).single()`
INSERT	`saveCertificateToSupabase`	11 fields; called **only after a real mined receipt**
UPDATE	`updateCertificateStatusInSupabase`	`status` only, constrained by `certificate_id` **and** the chain-derived `certificate_hash`
DELETE	—	**never called**

10.3 AUTHORITY SPLIT

Code comment, verbatim (`src/lib/certificates.ts:274-281`):

> *The blockchain is authoritative for certificate identity (certificate_id + certificate_hash) and status. Supabase rows are untrusted display metadata: with the public anon key in the browser, a row can only be trusted to the extent that it is anchored to an on-chain record. The helpers below implement that boundary; RLS itself is NOT enforced here (see supabase/rls-hardening.sql — requires external configuration).*

- **Blockchain = authoritative proof layer:** existence, hash, issuer, status, timestamp, tx/block evidence.
- **Supabase = metadata/display/index layer:** recipient, email, course, type, human date, cached tx hash (a *candidate* that is accepted only after receipt+log verification, `certificates.ts:401-410`), and dashboard listing.

Failure behaviour: if every row fails reconciliation, the dashboard falls through to `fetchAllCertificates()` from the chain rather than rendering an empty or poisoned dashboard (`certificates.ts:606-626`). [IV]

10.4 THE METADATA POISONING PROBLEM (TASK 9)

Four concrete weaknesses were found and are recorded in `context_md/2026-10-16_132041.md:27-37`:

1. No RLS represented in the repo → cannot be verified from the repository.
2. The browser could write every column with the anon key (policies unknown).
3. `/certificate/$id` rendered Supabase rows as “Verified” with no chain cross-check.
4. `syncCertificatesFromChain` was Supabase-first with no reconciliation.
5. `verifyCertificate` copied recipient/course/type from Supabase even when the fingerprint mismatched the chain anchor.
6. `updateCertificateStatusInSupabase` updated by `certificate_id` only, so a poisoned duplicate row could be flipped.

Application-side protections implemented (all additive, all app-side): `lookupOnChainCertificateRecord`, `isSupabaseMetadataConsistentWithChain`, `reconcileSupabaseRowsWithChain`, `fetchVerifiedCertificateFromSupabase`, metadata gating inside `verifyCertificate`, chain-first reconciliation in sync, and the dual-key status-update guard. Detailed in Section 17. [IV]

10.5 RLS STATUS — STRICT VERDICT

> **RLS hardening SQL EXISTS, but its application to the live project is NOT VERIFIED by the repository.**

Evidence: `supabase/rls-hardening.sql:7-10` — *"This file is a REVIEWABLE PROPOSAL, not an applied migration. The repository does not contain the live database schema or any existing RLS policies, so this must be reviewed and applied externally (Supabase SQL editor or a migration) by the project owner."*; `certificates.ts:280-281`; `test/supabase-integrity.test.ts:13-14` — *"Actual RLS policies live in the Supabase database and cannot be tested from this repository."* Negative evidence: only one file under `supabase/`, no `supabase/migrations/`, no CI, no captured applied-state output. [EXT]

What the proposal would do: enable RLS; public `SELECT` for `anon, authenticated`; constrained `INSERT` (`status in ('active','revoked')`, `certificate_id ~ '^CERT-[0-9]{4}-[0-9]{4,6}$'`, `certificate_hash ~ '^0x[0-9a-f]{64}$'`, `transaction_hash ~ '^0x[0-9a-fA-F]{64}$'`, `issuer_wallet ~ '^0x[0-9a-fA-F]{40}$'`, non-empty trimmed `ipfs_cid`); `UPDATE` restricted to `status` plus an immutability trigger `prevent_certificate_metadata_update()`; `DELETE` revoked. It also documents residual risks honestly (anon INSERT can still add format-valid rows; anon UPDATE can still flip `status`, mitigated because verification reads status from chain). [EXT]

---

11. Verification Method A — `verifyCertificate(hash)`

Function: `src/lib/certificates.ts:642-883`. Returns `{ outcome: "verified" | "revoked" | "not_found" | "error", hash, certificate?, checks, error? }`.

11.1 EXACT SEQUENCE

Stage	Name	Action	Data sources	On failure
0	Validate hash format	`hash.trim()`, then `isHashLike` = `/^0x[0-9a-fA-F]{64}$/`	none (offline)	`error` / `invalid_hash` — *"Invalid certificate hash. Expected a 0x-prefixed 64-character hexadecimal SHA-256 hash."*
1	Connect to blockchain	`getReadOnlyProvider()`	public Amoy RPC	`error` / `rpc_error` — *"Unable to connect to Polygon Amoy network…"*
2	Query smart contract	`getContract(provider)`	—	`error` / `contract_error` — *"Unable to load the CertiProof smart contract…"*
3	Search for certificate	requires `getCertificateCount()`, `getCertificate(string)`, `certificateIds(uint256)`; **linear scan** comparing `result[0].toLowerCase()` to the lowercased input (or its bytes32 form)	chain	interface missing → `contract_error`
3a	Enrich	on match: Supabase by hash → `resolveRegistrationTx` → `isAuthorizedIssuer(issuer)` → `getBlockNumber()`	Supabase + chain	per-check degradation, see below
4	Decision	`outcome = revoked ? "revoked" : "verified"`; no match → `not_found`	chain	outer catch → `error` / `network_error`

Stage labels shown in UI: `["Validating hash format","Connecting to Polygon Amoy","Querying smart contract","Retrieving certificate record"]`.

11.2 CHECKS, IN EXACT ORDER

#	Label	Value shown	`pass`	Proves
1	`Hash Match`	`PASS`	true (once reached)	the hash exists in the on-chain registry
2	`Metadata Fingerprint` *(only if a Supabase row exists)*	`MATCH` / `MISMATCH`	metadata gate result	the displayed metadata re-hashes to the chain anchor
3	`Issuer Authorization`	`AUTHORIZED` / `UNAUTHORIZED`	boolean	issuer in `authorizedIssuers` **at read time**
4	`Revocation Status`	`REVOKED` / `ACTIVE`	`!revoked`	contract `status`
5	`Blockchain Record`	`BLOCK #<n>` or `CONFIRMED`	receipt/tx recovered	registration tx recovered
6	`Confirmations`	`<n> blocks` or `PENDING`	`confirmations > 0`	`currentBlock - blockNumber > 0`
7	`IPFS Document`	`STORED` / `MISSING`	non-empty CID string	CID present on chain — **presence only, no content fetch or hash compare**

What Method A proves: existence, non-revocation, issuer authorization at read time, that any displayed metadata re-derives to the anchored hash, and that a registration transaction exists with ≥1 confirmation. **What it does not prove:** that the IPFS content still matches (not fetched), that the institution's claim is academically true, or that a check like "Issuer Authorization" affected the outcome — the outcome is derived solely from hash match + `status` (`certificates.ts:737, 850-855`). [IV] [OL]

11.3 STATUS IS CHAIN-ONLY

`status: revoked ? "revoked" : "active"` derives from `result[5]`, the contract tuple — Method A never trusts Supabase for status. Metadata (recipient/course) is copied from the row **only if** `isSupabaseMetadataConsistentWithChain(...)` passes; otherwise the result keeps `recipientName: ""`, `course: ""`. [IV]

---

12. Verification Method B — QR

12.1 PAYLOAD

Exact template emitted by the application: **`https://certiproof.app/verify/${hash}`** where `hash` is the metadata fingerprint (`src/components/certificate/CertificatePreview.tsx:277`). Benchmark mirror: `buildQrPayload(hash)` → `` `${QR_APP_BASE}/${hash}` `` with `QR_APP_BASE = "https://certiproof.app/verify"` (`scripts/bench-verification.ts:73-87`).

QR rendering: `QRCode.toDataURL(value, { width: size*2, margin: 1, color: { dark:"#1b2337", light:"#ffffff" } })` (`QRCodeCard.tsx:10-14`); standalone microbenchmark 136.43 ms.

12.2 EXTRACTION

`extractHashFromQrPayload(payload)` = last path segment before `/?#`, regex `/\/verify\/([^/?#]+)/`, returning `null` when absent. Null / malformed payloads produce `malformed_payload_no_hash_extracted` in the benchmark; a URL with a bad hash proceeds to Method A and yields `invalid_hash`. [IV]

12.3 AFTER EXTRACTION

Identical pipeline to Method A: the route `src/routes/verify_.$hash.tsx` calls `verifyCertificate(hash, …)`. A test asserts a malformed hash extracted from a QR payload flows through the same validation as Method A (`test/verification.test.ts:55-63`). **There is no independent Method B result object in-app — Method B is Method A after URL extraction.** [IV] [OL]

12.4 SCOPE LIMIT

No in-app camera scanner exists (`jsQR` / `getUserMedia` / `Html5Qrcode` — 0 matches); `src/routes/verify.tsx:144-168` is a descriptive placeholder whose copy reads *"Point a phone camera at the certificate QR code to scan the certificate hash."* Benchmark timing explicitly excludes camera decoding. The paper must say "QR-encoded verification URL" rather than "in-app QR scanning". [OL]

What Method B proves: exactly what Method A proves, reached via a machine-readable transport that eliminates copy/paste error. **Limitation:** a forged QR image pointing at a different fingerprint simply verifies a different (or non-existent) certificate; the QR carries no authority of its own.

---

13. Verification Method C — document upload (`analyzeDocument`)

Function: `src/lib/certificates.ts:1401-1599`. Stages: `["Reading document","Extracting certificate metadata","Searching blockchain registry","Comparing certificate fingerprint"]`. Outcomes: `"match" | "mismatch" | "not_found"`.

13.1 FLOW

1. **Stage 0:** `documentByteHash = "0x" + SHA256(file bytes)` — computed, reported, never compared.
2. **Stage 1:** `extractCertificateMetadata(bytes)`. If no `certificateId` → `not_found` with rows `Certificate ID: (not found in document) / NOT FOUND` and `Document byte hash: n/a – not the registry key`, **no network call**.
3. **Stage 2:** `chainCert = fetchCertificateFromChain(id)` (chain) and `registryCert = fetchCertificateFromSupabase(id)` (Supabase); errors degrade both to `null`. No chain record → `not_found`. No Supabase row → `not_found` with `registeredHash: chainCert.hash` and `Certificate fingerprint: (metadata unavailable – cannot recompute)`.
4. **Stage 3 — Fingerprint #1 (registry integrity):** recompute the fingerprint from the **Supabase row** metadata and compare to the chain hash → `registryFingerprintMatches`.
5. **Field comparisons:** `recipientMatches` (exact, trimmed), `courseMatches`, `dateMatches` (both sides through `normalizeIssueDate`). A `null` extracted field can never match.
6. **Fingerprint #2 (document identity):** recompute the fingerprint from the **extracted** fields — **only when all four are present** — and compare to the chain hash → `documentFingerprintMatches`. Incomplete metadata ⇒ `null` ⇒ can never pass.
7. **Outcome:** `match` iff `extracted.complete && recipientMatches && courseMatches && dateMatches && registryFingerprintMatches && documentFingerprintMatches`; otherwise `mismatch`.

Field rows rendered: `Certificate ID`, `Recipient`, `Course`, `Issue Date`, `Certificate fingerprint`, `Document metadata fingerprint`. [IV]

13.2 WHAT EACH METHOD PROVES — COMPARISON TABLE

Method	Input	Main process	Data sources	Detects	Limitations
A	`0x` +64 hex fingerprint	validate → read-only provider → linear registry scan → enrich → 7 checks	chain RPC, Supabase (metadata only, gated), event log/receipt	non-existent certificate; revoked certificate; unauthorized issuer; displayed metadata that does not re-hash	~6 s mean latency (O(n) scan); `eth_getLogs` unreliable on free RPC; IPFS check is presence-only; no document inspection
B	QR image → URL	extract fingerprint → identical to A	same as A	same as A	no in-app scanning; QR carries no authority; inherits all Method A limits
C	PDF file bytes	byte hash → label-anchored field extraction → chain + Supabase lookup → two fingerprint recomputations → field-by-field compare	PDF content stream, chain, Supabase	document whose printed recipient/course/date/ID differ from the anchored record; unknown ID; incomplete/absent metadata	text-PDF only (no OCR); layout-dependent (jsPDF label anchors); needs a Supabase row for the registry comparison; revoked certificates still report content `match` with `status: revoked`; byte hash never compared to anything

---

14. PDF Extraction and Tamper Detection (Task 7)

14.1 WHAT IS WRITTEN INTO THE PDF

`src/lib/pdf/generate-certificate.ts` writes **visible text-show operations with static labels** — it never calls `setProperties` / `setTitle`, so there are **no PDF Info/XMP metadata keys**. Exact literals:

```
"This is to certify that"            (anchor for recipient)
"has successfully completed"         (anchor for course)
"Date: <issueDate>"
"Certificate No: <certificateNumber>"
"ID: <certificateId>"
"Certificate of Completion"
"Scan to verify"                      (only when a QR image is embedded)
"This certificate is blockchain-verified…"   (footer)
```

Note: issuance calls the generator **without** `qrCodeDataUrl` (`certificates.ts:1030-1039`), so issued PDFs contain **no embedded QR**; the QR lives in the HTML `CertificatePreview`. [IV]

14.2 WHY THE ORIGINAL IMPLEMENTATION FAILED

`extractPdfFields` (old, since deleted) regex-scanned the latin1 stream for only CERT-ID and date patterns. Recipient and course were never extracted; `analyzeDocument` hardcoded `document: "(cannot extract)"` with `match: true`, so a modified recipient or course could not change the outcome. Measured consequence: **Task 6 recorded 2/4 tamper detection** — issue-date and certificate-ID detected; recipient-name and course returned `match` and were **not** detected (`benchmarks/verification-results.json` → `reliability.tamperDetected = "2/4 (50%)"`). Source: `context_md/2026-10-15_235500.md:12-16` and `context_md/2026-10-15_230829.md:68-70`. [EM] [OL]

14.3 HOW THE CURRENT EXTRACTOR WORKS

`src/lib/pdf/extract-certificate-metadata.ts` (211 lines):

1. **Decode** the bytes with `TextDecoder("latin1")`.
2. **Collect literal strings** in stream order with `/\(((?:[^()\\]|\\.)*)\)/g`.

3. **Unescape** PDF escapes: `\n \r \t \b \f \( \) \\`, line continuations (`\`+LF/CRLF), 1–3 digit octal → `String.fromCharCode(parseInt(oct,8))`, unknown escape keeps the character.
4. **Sanitize:** control chars `[\u0000-\u001F\u007F-\u009F]` → space; trim edges; empty → `null`; length > 300 → `null`; **internal spaces preserved** (collapsing them would create false mismatches in a domain where internal whitespace is significant).
5. **Label-anchored reads:**
   - `valueAfterLabel(lines, label)` — line starting with the label; value = remainder with leading `:`/whitespace stripped; empty → `null`.
   - `valueAfterAnchor(lines, anchor)` — first line *containing* the anchor; value = **the next line**, rejected if it matches `STATIC_LABEL_PATTERN`.
6. **Static-label rejection** prevents an empty recipient/course from absorbing the following label: `/^(this is to certify that|has successfully completed|certificate of completion|date:|certificate no:|id:|scan to verify|this certificate is blockchain-verified)/i`.
7. **Field resolution:** `certificateId` from `ID:` (full pattern `^CERT-\d{4}-\d{4,6}$`, else substring `CERT-\d{4}-\d{4,6}`), falling back to `Certificate No:`, then to the first `CERT-YYYY-NNNNNN` token anywhere; `recipientName` after *"this is to certify that"*; `course` after *"has successfully completed"*; `issueDate` from `Date:` **only if** it matches `DATE_VALUE_PATTERN` (ISO date, `12 Jan 2026`, `Jan 12, 2026`, or `12/12/2026`).
8. **Missing fields return explicit `null` — never a placeholder, never a throw.**
9. **`complete = Boolean(certificateId && recipientName && course && issueDate)`** — true iff all four are non-null.

Documented limitations in the module header: scanned/image-only PDFs yield all-null + `complete=false`; non-CertiProof layouts are unsupported; **no OCR**. [IV]

14.4 TAMPER CASES — OFFLINE TEST MATRIX (`TEST/METHOD-C-EXTRACTION.TEST.TS`, 14 TESTS)

#	Test name (verbatim)	Expectation
1	`1+2+3+4. extracts certificate ID, recipient name, course and issue date`	all four match base values
2	`5. all four fields extracted from one PDF (complete=true), including with an embedded image`	`complete === true` with a 1×1 PNG
3	`6. recomputed fingerprint from extracted metadata equals the canonical fingerprint of the input metadata`	round-trip equality
4	`7. recipient-name modification changes extraction and alters the recomputed fingerprint`	value becomes `Wrong Person`, fingerprint differs
5	`8. course modification changes extraction and alters the recomputed fingerprint`	value becomes `Wrong Course`, fingerprint differs
6	`9. issue-date modification changes extraction and alters the recomputed fingerprint`	date → `2026-01-01`, fingerprint differs
7	`10. certificate-ID modification changes extraction and alters the recomputed fingerprint`	id → `CERT-2099-999999`, fingerprint differs
8	`11. missing recipient name → extraction incomplete (explicit, not a placeholder)`	`recipientName === null`, `complete === false`
9	`12. missing course → extraction incomplete (explicit, not a placeholder)`	`course === null`, `complete === false`
10	`13. malformed PDF bytes do not crash the extractor and report incomplete extraction`	512 random bytes → all null, `complete === false`
11	`14. empty file does not crash the extractor or the verifier`	empty → `complete === false`; `analyzeDocument(empty).outcome === "not_found"`
12	`15. partial extraction: an ID-like line without anchors yields incomplete metadata`	id extracted, others null, `complete === false`
13	`names with parentheses and escapes survive extraction (PDF string escaping)`	`"Ana (María) O'Brien"` round-trips
14	`ISO datetime issue dates printed by issuance normalize consistently`	extracted datetime's fingerprint equals the plain-date fingerprint

Related coverage: 4 dedicated `TEST:` mutation tests in `certificate-fingerprint.test.ts`, 7 tamper/poisoning tests in `supabase-integrity.test.ts`, 3 offline Method C early-return tests in `verification.test.ts`. [IV]

14.5 LIVE METHOD C RESULTS

`scripts/live-method-c-check.ts` on target `CERT-2026-233459` (issueDate `2026-10-15T00:00:00+00:00`): **A original → `match` (all 6 checks true, 741 ms); B recipient → `mismatch`; C course → `mismatch`; D date → `mismatch`; E unknown ID → `not_found`. 5/5 PASS, first run** (`context_md/2026-10-15_235500.md:45-48`). [EM]

14.6 WHAT METHOD C CAN AND CANNOT DETECT

Can detect: a change to the printed recipient name, course, issue date, or certificate ID in a CertiProof-generated text PDF, because the change either alters the recomputed fingerprint or makes the ID unknown. Also: missing fields (`complete=false` ⇒ fingerprint can never pass), malformed and empty files (`not_found`, no crash). [IV]

Cannot detect: visual/graphical tampering outside the extracted text fields (logos, signatures, grades not in the four fields); scanned or image-only PDFs (no OCR); text changed in a way that still yields the same four fields; tampering where an attacker also forges a matching record — impossible without chain access, which is the point, but must be stated as the threat model; and certificates with no Supabase row (`not_found` with "metadata unavailable"). The session note is explicit: *"Do not claim arbitrary visual-tampering detection: scope is metadata tampering in CertiProof-generated text PDFs"* (`context_md/2026-10-15_235500.md:64`). [OL]

---

15. Transaction Confirmation State Machine (Task 4)

Module: `src/lib/blockchain/confirmation.ts` (344 lines).

15.1 STATES AND REASON CODES

```
type TransactionConfirmationState = "confirmed" | "failed" | "pending";

type FailedConfirmationReason =
  | "mined_reverted" | "receipt_status_unknown" | "submission_error" | "replacement_cancelled";

type PendingConfirmationReason =
  | "timeout_no_receipt" | "wait_error_no_receipt" | "receipt_not_mined";
```

`confirmed` carries `reason: "mined_success"` , `txHash` , `receipt` ; `failed` carries `txHash: string | null` (null only for `submission_error`); `pending` carries `txHash` and `receipt: null` . [IV]

15.2 RECEIPT VALIDATION RULES

```
isValidReceipt(r): number.isFinite(r.blockNumber) && r.blockNumber > 0
classifyReceipt:
   status === 1  → confirmed / mined_success
   status === 0  → failed    / mined_reverted
   status === null → failed  / receipt_status_unknown   // mined, but never "confirmed"
```

The `blockNumber > 0` gate exists specifically because a synthetic `{hash, blockNumber: 0, status: 1}` receipt is never valid evidence of mining. [IV]

15.3 WHY THE OLD IMPLEMENTATION WAS UNSAFE

Before Task 4, `issueCertificate` / `revokeCertificate` caught the 60 s `tx.wait()` timeout and fell back to a synthetic `{ hash, blockNumber: 0, status: 1 }` receipt — **reporting SUCCESS while the transaction was still pending**. Measured: **5 of 10** Task-3 benchmark runs hit the timeout, and those transactions were mined **37–166 s after** `issueCertificate` had already returned (`context_md/2026-10-15_184924.md:7` , `2026-10-14_231247.md:43-49`). The ~60 s "confirmation" values in that era were the timeout duration, not receipt wait. Fabricated `blockNumber: 0` receipts were dangerous precisely because they are structurally indistinguishable from success to a caller that does not validate. [EM] [OL]

15.4 TIMEOUT, FINAL CHECK, REPLACEMENT, CLEANUP

- `CONFIRMATION_TIMEOUT_MS = 60_000` , comment: *"Measured baseline from Task 3. Do not change without a measurement task."* A test pins `expect(CONFIRMATION_TIMEOUT_MS).toBe(60_000)` .
- Timeout ⇒ **one final receipt check**; if found, classify from it; else `pending / timeout_no_receipt` . Explicitly *"TIMEOUT ≠ FAILED and TIMEOUT ≠ SUCCESS"*.
- `waitForTransactionConfirmation` **never throws**; internal errors degrade to `pending / wait_error_no_receipt` .
- Receipt lookup uses the read-only public RPC first, wallet provider second, and returns `null` on failure — *"never fabricates one"*.
- Timer cleared in `finally` on success, failure, and timeout (3 dedicated tests via a recording schedule seam).

- Replacement: `TRANSACTION_REPLACED` with `cancelled === true` → `failed / replacement_cancelled`; a repriced replacement that mined successfully **confirms under the mined hash**.
- `submitTransactionAndConfirm`: submission throws or yields no `tx.hash` → `failed / submission_error` with `txHash: null`; `onSubmitted({hash})` fires before the wait.

15.5 PENDING REGISTRY AND ONE-SHOT RE-CHECK

`src/lib/pending-transactions.ts` — in-memory `Map<txHash, record>`; kinds `issuance` (with `certificateId`, `fingerprint`, `ipfsCid`, `issuerWallet`, `input`, `submittedAt`) and `revocation`. Header: *"the session-lifetime mechanism … a page reload loses the payload … Pending records are NEVER written to Supabase."* [OL]

`getTransactionState` / `checkPendingTransaction` do **one** RPC lookup (no polling): valid receipt + `status 1` → `confirmed`; `status !== 1` → `failed`; no/invalid receipt → `pending`; **RPC error → `pending`, never success**. On later `confirmed` for an issuance record, the certificate is built from the stored fingerprint, added to the store, and only then persisted to Supabase — *"Persist only now — the certificate has a real mined receipt"*. [IV]

15.6 THE 29 CONFIRMATION TESTS

Group (describe)	Count	Purpose
confirmation baseline	2	60 s timeout pinned; `blockNumber: 0` never valid
CASE A real successful receipt	1	submitted + real receipt → `confirmed`
CASE B failed receipt	3	`status 0` → failed; `CALL_EXCEPTION` revert → failed; mined-without-success-status never confirmed
CASE C timeout without receipt	3	timeout → `pending`, MUST NOT be success; synthetic `blockNumber: 0` in wait path and final-check path never confirmed
CASE D/E wait errors	5	tx hash preserved; late receipt → confirmed; receipt after wait error classified from receipt; cancelled replacement → failed; repriced replacement → confirmed under mined hash
timer cleanup	3	timer cleared on success, failure, timeout
`submitTransactionAndConfirm`	4	submission throw → `submission_error`; no hash → `submission_error`; `onSubmitted` ordering; hash survives timeout
`getTransactionState` one-shot	5	status 1 → confirmed; status 0 → failed; no receipt → pending; **RPC error → pending**; synthetic `blockNumber: 0` → pending
pending registry	3	record/preserve/remove; revocation removed after definitive re-check; reset
Total	**29**	

[IV]

16. Wrong-Network Protection (Task 8)

16.1 THE ORIGINAL VULNERABILITY

`issueCertificate` and `revokeCertificate` had **no pre-signing network check**. Signing on a non-Amoy chain sends a real transaction that mines as a **no-op with receipt status 1** at an address without code. The confirmation machine would classify that as `confirmed / mined_success`, producing a false “anchored” or “revoked” result, wasting gas, and (for issuance) wasting an IPFS pin. Recorded in `context_md/2026-10-16_002500.md:33-37` and in the code comment at `certificates.ts:986-990`. [OL]

16.2 THE GUARD

```
export async function isConnectedToAmoy(): Promise<boolean> {
  try { return (await getChainId()) === POLYGON_AMOY.chainId; }   // 80002
  catch { return false; }
}
```

- Issuance guard: right after timer creation, **before** fingerprint / PDF / IPFS / signing (`certificates.ts:986-1002` , call at 991).
- Revocation guard: before `getSigner()` (`certificates.ts:1203-1210` , call at 1205).
- Throws typed `IssuanceError { type: "wrong_network", message: "Wrong network. Please switch MetaMask to Polygon Amoy before issuing." }` and records a failure with stage `blockchainPreparation` , `transactionSubmitted: false` , `transactionHash: null` .
- Wallet-level guard also exists in `useWallet.tsx:62-68` (`"Wrong network. Please switch to Polygon Amoy."`), and `switchToAmoy()` offers `wallet_switchEthereumChain` / `wallet_addEthereumChain` .

16.3 WHY THE GUARD MUST PRECEDE FINGERPRINT/PDF/IPFS/SIGNING

Because side effects before the guard are irreversible or costly: IPFS pinning spends quota, and any submitted transaction can mine as a successful-looking no-op. Placing the check first guarantees **zero external side effects** on the wrong chain. [IV]

16.4 TEST RESULT

`scripts/wrong-network-check.ts` (zero transactions, zero funds): shim reports `eth_chainId 0x1` ; both `issueCertificate` and `revokeCertificate("CERT-2026-000000")` throw typed `wrong_network` ; **only** `eth_chainId` **appears in the recorded RPC call list** (no `eth_sendTransaction`); failure record has `transactionSubmitted=false` , `transactionHash=null` , `errorCategory=wrong_network` . **ALL PASS, zero funds spent** (`context_md/2026-10-16_152855.md:26-29`). [EM]

Gap: the guard is exercised by a script, not by an automated unit test — no test in `test/` covers the issuance or revocation path (Task 8 explicitly notes “no tests call issuance/revoke paths”). [OL]

---

17. Supabase Integrity Hardening (Task 9)

All in `src/lib/certificates.ts` unless noted. All additive; contract, fingerprint protocol, QR, IPFS, confirmation machine, and Methods A/B/C algorithms were untouched.

17.1 THE SEVEN PROTECTIONS

#	Protection	Function / locus	Mechanism
1	**verifyCertificate metadata gating**	`certificates.ts:740-744, 796-820`	recipient/course/type copied from the row only when `isSupabaseMetadataConsistentWithChain(row, onChainHash)` ; a `Metadata Fingerprint: MATCH/MISMATCH` check row is always reported
2	**Certificate detail chain cross-check**	`fetchVerifiedCertificateFromSupabase` (`401-426`) + `src/routes/certificate.$id.tsx:42-57, 94`	returns `null` if no row, no chain cert, or `row.hash !== chainCert.hash` ; metadata overlaid only when trusted — *"a metadata-only Supabase row never renders as a verified certificate"*
3	**Row reconciliation**	`reconcileSupabaseRowsWithChain` (`369-385`) with injected lookup	`missing` → dropped; hash mismatch → dropped; found+match → kept with **chain hash and chain status overriding**; `error` → **row kept unchanged** (availability during RPC outages)
4	**Status update constraint**	`updateCertificateStatusInSupabase` (`246-272`)	requires `lookupOnChainCertificateRecord(...) === "found"` (missing **or** RPC error ⇒ skip, fail-closed), then `.eq("certificate_id", certId).eq("certificate_hash", chainHash)`
5	**Fingerprint consistency check**	`isSupabaseMetadataConsistentWithChain` (`345-356`)	pure recompute: `generateCertificateFingerprint({certificateId, recipientName, course, issueDate}).toLowerCase() === onChainHash.trim().toLowerCase()`
6	**Chain-first dashboard sync**	`syncCertificatesFromChain` (`606-626`)	reconciles first; if every row fails, falls through to `fetchAllCertificates()` from the chain
7	**Method C double fingerprint**	`certificates.ts:1493-1540`	registry metadata re-hash vs chain **and** extracted metadata re-hash vs chain; incomplete metadata ⇒ `null` ⇒ can never pass

17.2 THE RE-KEYING ATTACK

An attacker inserts a row reusing a **real** `certificate_id` but carrying different metadata, or re-keys an existing row to another id. Defences: fingerprint gate (the metadata will not hash to the chain anchor), reconciliation (a row whose `certificate_hash` differs from the on-chain record for that id is dropped), and the dual-key status update (the poisoned row's hash will not equal the chain hash, so no update lands on it). Tests: `row re-keyed to a different certificate id no longer matches the on-chain anchor` , `row whose certificate_hash does not match the on-chain record is dropped` , `metadata-only row without an on-chain record is dropped` , `a fingerprint belonging to a different certificate does not match` . [IV]

17.3 THE POISONED-STATUS ATTACK

An attacker flips `status` directly (anon UPDATE). Defences: **verification never reads status from Supabase** — Method A reads `result[5]` , Method C takes `certificate.status` from the chain record, dashboard sync overrides status from the chain, and `updateCertificateStatusInSupabase` refuses on inconclusive identity. The residual risk is acknowledged in `rls-hardening.sql:37-47` : an anon client can still flip a status *in the database*, but the display layer will override it. Marked **[EXT]** because true prevention needs RLS or server-side writes. [IV] [EXT]

17.4 CHAIN-ONLY BEHAVIOUR AND RPC-ERROR BEHAVIOUR

- Chain-only certificates (on-chain, no Supabase row): Method A → `verified` with metadata `absent` and empty recipient; Method C → `not_found` with the explicit `metadata unavailable — cannot recompute` marker. Measured live (`context_md/2026-10-15_230829.md:71-73`). [EM]

- RPC error during reconciliation → row **kept** (fail-open, availability). RPC error during a **write** → update **skipped** (fail-closed). RPC error during confirmation re-check → `pending`, never success. This asymmetry is deliberate and test-pinned (`inconclusive lookup (RPC error) keeps the row unchanged — availability over strictness`). [IV]

17.5 WHY BLOCKCHAIN STAYS AUTHORITATIVE EVEN WITH MALICIOUS METADATA

Because no displayed field and no status decision can be produced by the database alone: displayed metadata must re-hash to the chain value, rows that do not anchor are dropped, status is read from the contract tuple, transaction hashes are accepted only after receipt + log verification (`resolveRegistrationTx`, `certificates.ts:106-161`), and writes are gated on chain identity. The database can therefore *withhold* or *corrupt* information (an availability attack), but it cannot *manufacture* a verified certificate. That distinction is the correct claim for the paper — and its converse ("the system is unavailable if Supabase is down for Method C") must also be disclosed. [IV] [OL]

---

18. Security / Threat Model

Derived **only** from the actual implementation. "Existing defense" names real code; "Result" states what was measured or proven by inspection; "Remaining risk" is honest.

Threat	Attack	Existing defense	Result	Remaining risk
Certificate forgery (new identity)	Attacker fabricates a PDF and computes any hash	`verifyCertificate` linear-scans the chain registry; unknown hashes → `not_found`	5/5 unknown hashes → `not_found` [EM]	Latency ~10 s for a miss; no cryptographic proof-of-absence beyond the scan
Certificate forgery (existing identity)	Attacker copies a real PDF	Fingerprint already anchored; the copy *is* authentic — that is correct behaviour	10/10 authentic → `verified` / `match` [EM]	A copied genuine certificate remains genuine; only revocation changes its status
PDF metadata tampering	Edit recipient/course/date/ID in the PDF	Method C re-extracts fields and recomputes the fingerprint	Live 5/5 (recipient/course/date → `mismatch`, unknown ID → `not_found`) [EM]	Text-PDF only, no OCR; **stored benchmark JSON still records the pre-fix 2/4** [OL]
Certificate ID manipulation	Change `ID:` to another valid-looking ID	Falls to `not_found` for an unregistered ID; for a registered ID the other three fields will mismatch	Measured `not_found` [EM]	—
Hash manipulation	Present a modified hash	Format gate `/^0x[0-9a-fA-F]{64}$/` before any RPC	4/4 malformed → `invalid_hash` in ≤0.23 ms [EM]	—
QR manipulation	Re-point a QR to another URL	QR payload is only a transport; extracted hash goes through identical validation	Equivalence test + malformed-QR probes [IV] [EM]	A QR pointing at a *valid other* certificate verifies that other certificate — inherent to the design
Supabase poisoning	Insert/re-key/flip rows with the anon key	7 protections, Section 17	14 offline tests; live 24/24 anchored reconciliation [EM]	RLS not applied ⇒ DB-level writes still possible [EXT]
Wrong-network transaction	Sign while MetaMask is on another chain	`isConnectedToAmoy()` guard before any side effect	Live: only `eth_chainId` observed, 0 funds spent [EM]	Guard is script-tested only, not unit-tested [OL]
Unauthorized issuer	Unregistered wallet calls `registerCertificate`	`onlyAuthorizedIssuer` at the contract	Contract-level `require` (inspection)	**No automated contract test exists** [OL]
Transaction replacement	Wallet replaces/cancels a tx	`TRANSACTION_REPLACED` handling: cancelled → failed; repriced+mined → confirmed under mined hash	2 dedicated tests [IV]	—
RPC failure	Public RPC down or throttled	Read paths degrade to `null` / `pending` / `error`; reconciliation fail-open; write path fail-closed	Tests for each branch [IV]	Verification unavailable during outages [OL]
IPFS failure	Pinata outage or quota exhaustion	Typed safe errors; issuance aborts before the chain call	4/4 error probes behaved as expected, 0 secret leakage [EM]	Issuance blocked while Pinata is down; no alternative pinning provider [OL]
IPFS quota abuse	Anyone POSTs to the upload server function	10 MB cap, MIME allowlist, empty-file rejection	Validation proven by 23 tests	**No authentication and no rate limiting** — the server function is publicly callable [OL] [EXT]

Threat	Attack	Existing defense	Result	Remaining risk
Credential leakage (Pinata)	Extract the JWT from the client	Server-only module + `VITE_` prefix removed + bundle scans	0 findings in client bundle scans [EM]	If the token was ever in a deployed bundle, rotation status is **not recorded** [EXT]
Credential leakage (deployer key)	Read the key from the repo	`.env` gitignored; `DEPLOYER_PRIVATE_KEY` referenced only by Node tooling; hardhat has an **empty** fallback and deploy exits if unset	Containment-checked scans found no values in `.output`, `benchmarks`, `test`, `scripts` [EM]	`.env` exists locally with real values; rotation policy undocumented [EXT]
Client bundle secret leakage	Inspect built assets	Static scan patterns in `bench-verification.ts:886`; Task 8/10 scans	`findings: []` [EM]	No CI to enforce the scan on every build [OL]
Malformed input	Garbage files, weird names, unicode	Extractor never throws; upload validation; React escaping	3/3 garbage inputs → `not_found` in ≤0.56 ms; unicode hashing tested [EM] [IV]	`chart.tsx` uses `dangerouslySetInnerHTML` with developer-controlled config only (Task 8, area E) [OL]
XSS	Inject script via certificate fields	React default escaping; no `dangerouslySetInnerHTML` on user data	Task 8 area E verdict "VERIFIED SAFE" (inspection)	No automated XSS test [OL]
Replay / duplicate registration	Register the same certificate twice	`"Certificate already exists"` require on empty `ipfsCID`	Contract-level (inspection)	Existence keyed on the CID string rather than an explicit flag [OL]
Revocation manipulation	Revoke someone else's / re-revoke	`onlySuperAdmin` + `"Already revoked"`; **irreversible in code**	Contract-level (inspection) + 1 live revoke [EM]	Single immutable super admin, no 2-step, no pause [OL]
Database manipulation	Delete/alter rows	No `DELETE` path in app code; proposed RLS revokes DELETE	Proposal only	Not enforceable from the repo [EXT]
Dashboard misreporting	Treat issuance as verification	`metrics.verified = certificates.filter(c => c.chain.transactionHash).length` (`dashboard.index.tsx:55`)	**Confirmed still present**	The "Verified" metric counts records with a tx hash, **not verification events** — must be labelled honestly [OL]
Frontend route access	Direct URL to `/dashboard`	`dashboard.tsx` is a pure layout shell with **no auth guard**; `login.tsx` checks `checkIssuerAuth` but is advisory	Inspection	Frontend guards are UX controls, not authorization; contract remains the enforcement layer [OL]

CONFIDENTIAL

19. Experimental Setup

19.1 ENVIRONMENT (AS RECORDED IN THE BENCHMARK JSONS)

Parameter	Value	Where recorded
Network	Polygon Amoy TESTNET	`issuance-results.json meta.network`, `verification-results.json meta.network`
Chain ID	80002	`issuance-results.json meta.chainId`
RPC	`https://polygon-amoy-bor-rpc.publicnode.com` (public, shared)	`verification-results.json meta.rpcUrl`
Contract	`0x09921D25cB513793e17082Cce0fB4F520f0CDF50`	all three JSONs
Issuer wallet	`0x3e72b748a543DEc762a297F7304A04F980d7bC01`	`issuance-results.json meta.issuerAddress`
IPFS gateway	`https://gateway.pinata.cloud/ipfs`	`ipfs-results.json meta.gateway`
Pinning API	`https://api.pinata.cloud/pinning/pinFileToIPFS` (+ `/unpin/{cid}`)	`scripts/bench-ipfs.ts:67-68`
Supabase	app client, `certificates` table	app libs
Runtime	Bun	`package.json`
Date range of measurements	2026-10-15 (IPFS, verification) → 2026-10-16 (issuance)	`meta.generatedAt`

19.2 HARNESS DESIGN

- **`bench:issuance`** (`scripts/bench-issuance.ts`): runs the **real** `issueCertificate()` headlessly; `DEPLOYER_PRIVATE_KEY` is provided through an EIP-1193 `window.ethereum` shim that signs locally, so MetaMask is substituted without changing app code; the TanStack server-function transport bridge is rebuilt in-process, so only the browser→server HTTP hop is absent; real Amoy transactions and real Pinata pins are performed. Preflight aborts unless `DEPLOYER_PRIVATE_KEY`, `VITE_CONTRACT_ADDRESS`, `PINATA_JWT`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` are present, the RPC reports 80002, the wallet is an authorized issuer, and the balance is ≥ 0.05 POL. **No warm-up phase; 50 ms inter-run pause.**
- **`bench:ipfs`**: 7 sizes × 5 runs with mulberry32-seeded unique synthetic content, per-cycle upload → gateway GET (3 attempts, 2500 ms backoff, 60 s timeout) → SHA-256 integrity → unpin; plus a real certificate PDF, a delayed re-retrieval, a deduplication phase, and four live error probes.
- **`bench:verification`**: read-only discovery of the live registry; 10 runs per method; negatives (5 unknown hashes, 4 malformed hashes, 4 malformed QR payloads, 2 unknown-QR hashes); 4 tamper cases via equal-length latin1 byte replacement; 3 garbage inputs; chain-only and revoked probes; fingerprint consistency over all Supabase-backed certs; and a static secret scan. 300 ms pause between calls; discovery retried 4× with 150 ms pacing.

19.3 TIMING BOUNDARIES (AS STORED IN THE JSONS)

Issuance: `totalIssuanceMs` excludes the fire-and-forget Supabase insert; confirmation = `tx.wait()` raced against the 60 s app timeout, where timeout-without-receipt is a **failed** `confirmation_pending`, never success. Verification: `methodA` stages `validate / provider / contract / searchAndDecision`; `methodC` stages `readAndByteHash / extractFields / registrySearch / compareAndDecision`; Method B excludes camera decoding. IPFS: `uploadMs` from `uploadToIPFS(file)` to CID return; `retrievalMs` from gateway GET to full body read.

19.4 STATISTICAL CONVENTIONS

`stdDev` is the **sample** standard deviation (n−1). Failed runs are excluded from timing statistics but retained in raw/reliability blocks. With n=1 the stored `stdDev: 0` is a coding convention, **not a measured zero variance** — must not be presented as such. [OL]

19.5 WHAT IS NOT RECORDED

Machine/host specifications (CPU, RAM, OS, browser), Bun/Node versions, warm-up behaviour, and a commit hash per run are **not stored in any benchmark JSON**. Provenance rests on `meta.generatedAt` alone. Mark **NOT MEASURED / NOT RECORDED**. [OL]

---

20. Performance Results

20.1 ISSUANCE — `BENCHMARKS/ISSUANCE-RESULTS.JSON` (GENERATED 2026-10-16T08:11:24.156Z)

> **Critical caveat:** the stored file contains `runsRequested: 1`, so **every issuance statistic below has n = 1**. This single run (certificate `CERT-2026-273168`, Task 10) **overwrote the Task 3 N=10 baseline**, whose raw data no longer exists in the repository. See Section 35, discrepancy D-07.

Stage	Value (ms)
fingerprintGenerationMs	1.74
pdfGenerationMs	33.55
ipfsUploadMs	2117.72
blockchainPreparationMs	38.5
blockchainSubmissionMs	1897.70
blockchainConfirmationMs	5232.65
supabasePersistenceMs (excluded from total)	1468.89
totalIssuanceMs	**9328**
qrGenerationMs (microbenchmark, outside pipeline)	136.43

Transaction `0x823d7bc4bc50e18e4a43ea857f09ce0ab9c06f24fa1b4b9644be8a655788a703`, CID `QmVgzPmUdtTue47jgzwR1G3WogXuH9fgguoEgzRthmqoSu`, receipt status 1, block #50315729.

Superseded N=10 baseline (Task 3, 2026-10-14) — prose only, not in any JSON: fingerprint 4.67, pdf 14.67, ipfs 1524.84, prep 34.36, submit 1511.28, **confirm 44972.59**, supabase 456.29, **total 48063.75 ms**, 10/10 successful real issuances (`context_md/2026-10-14_231247.md:51-52`). That confirmation mean is inflated by the 5 runs that hit the 60 s timeout under the *pre-Task-4* code, so it is **not comparable** to the 5232.65 ms confirmation of the current implementation. **The paper must either re-run the issuance benchmark with n≥10 under current code or present only the n=1 value with an explicit caveat.** [EM] [OL]

20.2 VERIFICATION — `BENCHMARKS/VERIFICATION-RESULTS.JSON` (GENERATED 2026-10-15T17:36:44.832Z)

Population at measurement time: **28 on-chain, 23 Supabase rows, 23 Supabase-backed, 5 chain-only, 5 revoked.**

Metric	n	min	median	mean	max	stdDev
Method A total (ms)	10	2707.02	6097.00	6022.47	8006.71	1678.63
— stage `validate`	10	0.05	0.17	0.39	2.24	0.66
— stage `provider`	10	0.05	0.11	0.20	0.94	0.26
— stage `contract`	10	1.21	2.35	2.24	3.45	0.68
— stage `searchAndDecision`	10	2702.03	6092.99	6019.64	8005.06	1679.50
Method B total (ms)	10	2723.02	6062.79	6038.49	7977.36	1713.07
Method C total (ms)	10	515.15	544.20	551.11	624.44	32.66
— stage `registrySearch`	10	514.46	541.23	549.39	623.68	33.30
— stage `readAndByteHash`	10	0.16	0.34	1.34	5.23	1.99
— stage `extractFields`	10	0.02	0.04	0.07	0.20	0.06
— stage `compareAndDecision`	10	0.17	0.26	0.32	0.61	0.15

Negatives: unknown hash → `not_found` 5/5 in 10015–10926 ms; malformed hash → `invalid_hash` 4/4 in ≤0.23 ms; unknown QR hash → `not_found` 2/2 in ~10.2–10.6 s; garbage inputs (text-as-PDF 35 B, PNG 12 B, empty 0 B) → `not_found` 3/3 in ≤0.56 ms.

Edge cases: chain-only `CERT-2026-361028` → Method A `verified` (1617.05 ms, metadata `absent`, recipient empty), Method C `not_found` (532.64 ms, `metadataUnavailableIndicated: true`); revoked → A `revoked` 3364.66 ms, B `revoked` 3260.29 ms, C `match` + `certificateStatus: "revoked"` 594.85 ms.

Fingerprint consistency: **23/23 all-three-equal** (recomputed = on-chain = Supabase), 0 mismatches. Security scan: `findings: []`.

Dominant latency contributor: the linear on-chain registry scan — `searchAndDecision` is ~99.9% of Method A total and `registrySearch` is ~99.7% of Method C total. [EM]

20.3 IPFS — `BENCHMARKS/IPFS-RESULTS.JSON`

Full tables in Section 9.6. Headline: upload overall n=36 mean 9508.25 / median 3881.22 / min 1070.15 / max 54534.18 / stdDev 12037.31 ms; retrieval overall n=36 mean 7181.63 / median 7045.28 / min 4593.96 / max 11009.05 / stdDev 1687.95 ms; 39/39 uploads, 36/36 first-attempt retrievals, 36/36 byte-identical. [EM]

20.4 UNIFIED PERFORMANCE TABLE

Operation	n	min	median	mean	max	stdDev (ms)	Source
Fingerprint (SHA-256 canonical)	1	1.74	1.74	1.74	1.74	0 (n=1)	issuance JSON
— micro (SHA-256, N=1000)	1000	—	—	**0.264**	—	—	`context_md/2026-10-14_171533.md:34` (prose only, stale)
PDF generation (jsPDF)	1	33.55	33.55	33.55	33.55	0 (n=1)	issuance JSON
— jsPDF ×5 micro	5	—	—	**11.0**	—	—	prose, stale
QR generation	1	—	—	136.43	—	—	issuance JSON
IPFS upload	36	1070.15	3881.22	9508.25	54534.18	12037.31	ipfs JSON
IPFS retrieval	36	4593.96	7045.28	7181.63	11009.05	1687.95	ipfs JSON
Blockchain submission	1	1897.70	1897.70	1897.70	1897.70	0 (n=1)	issuance JSON
Blockchain confirmation	1	5232.65	5232.65	5232.65	5232.65	0 (n=1)	issuance JSON
Supabase persistence (non-blocking)	1	1468.89	1468.89	1468.89	1468.89	0 (n=1)	issuance JSON
Total issuance	1	9328	9328	9328	9328	0 (n=1)	issuance JSON
Verification A	10	2707.02	6097.00	6022.47	8006.71	1678.63	verification JSON
Verification B	10	2723.02	6062.79	6038.49	7977.36	1713.07	verification JSON
Verification C	10	515.15	544.20	551.11	624.44	32.66	verification JSON
Verification A miss (unknown hash)	5	10015.06	—	~10361	10926.37	—	verification JSON (mean DERIVED)

CONFIDENTIAL

20.5 ACCURACY / CORRECTNESS FIGURES

Measure	Value	Source
Method A valid → `verified`	10/10 (100%)	verification JSON
Method B consistent with A	10/10 (100%)	verification JSON
Method C valid → `match`	10/10 (100%)	verification JSON
Unknown hash → `not_found`	5/5 (100%)	verification JSON
Malformed hash rejected	4/4 (100%)	verification JSON
Tamper detected (pre-Task-7 benchmark)	**2/4 (50%)**	verification JSON — **stale**
Tamper detected (post-Task-7 live)	5/5 (100%) on the live Method C script	`context_md/2026-10-15_235500.md` — **prose only, n=1 per case**
Fingerprint consistency (recompute=chain=DB)	23/23 (100%)	verification JSON
Fingerprint compatibility audit	24/24 stored + 24/24 chain MATCH	`context_md/2026-10-16_152855.md` — **prose only**
IPFS byte integrity	36/36 (100%)	ipfs JSON
Blockchain-record (getLogs) check	**0/10 (0%)** at measurement time	verification JSON — **stale; fixed 2026-10-16**

The single most important honesty point for the paper: the two headline tamper-detection numbers pull in opposite directions (50% in the stored benchmark, 100% in the later live run) because the benchmark was run *before* the extractor rewrite. Neither number should be quoted without its date and context. See Section 35, D-05. [EM] [OL]

---

21. Live Experiments

Experiment	Objective	Environment	Input	Procedure	Result
Task 3 — live issuance	Instrument and measure the real issuance pipeline	Amoy, contract `0x0992…DF50`, issuer `0x3e72…7bC01`, balance 50.29 POL before run	synthetic bench recipients	10 real `issueCertificate()` runs through the EIP-1193 shim	**10/10 successful real transactions**; bimodal confirmation (5 genuine in 5–43 s, 5 hit the 60 s timeout and were assumed-success while mining 37–166 s later)
Task 4 — pending → confirmed	Prove pending ≠ success on a real transaction	Amoy, funded wallet	one 0-value self-transfer	`waitForTransactionConfirmation` with `TINY_TIMEOUT_MS = 1` → assert `pending` / `timeout_no_receipt` with hash preserved → one-shot `getTransactionState` polls every 5 s up to 180 s	hash `0xe056803f…` → **pending**, then **confirmed at block 50247727 at t+5.4 s**, `blockNumber > 0`, status 1
Task 5 — IPFS benchmark	Measure reliability, integrity, dedup, latency	Pinata API + `gateway.pinata.cloud`	1 certificate PDF + 7 sizes × 5 runs + dedup + 4 error probes	upload → gateway GET → SHA-256 compare → unpin; delayed recheck after ~653 s	**39/39 uploads, 36/36 first-attempt retrievals, 36/36 byte-identical, delayed integrity still true, dedup CID equality true, 4/4 error probes as expected, 0 secret leaks**
Task 6 — verification benchmark	Measure Methods A/B/C, tampering, negatives	Amoy read-only, 28-cert registry	10 targets/method + negatives + 4 tamper cases	full pipeline per method	A 10/10 verified (mean 6.02 s), B 10/10 consistent, C 10/10 match (mean 0.55 s), unknown 5/5, malformed 4/4, **tamper 2/4**, fingerprint 23/23

Experiment	Objective	Environment	Input	Procedure	Result
Task 7 — live Method C	Prove the rewritten extractor detects tampering	Amoy + Supabase, target `CERT-2026-233459`	authentic + 3 tampered variants + unknown ID	regenerate PDFs, run `analyzeDocument`	original → `match` (all checks true, **741 ms**); recipient → `mismatch`; course → `mismatch`; date → `mismatch`; unknown ID → `not_found`. **5/5 PASS**
Task 8 — wrong-network	Prove no signing/submission on the wrong chain	shim reporting `eth_chainId 0x1`	issue + revoke calls	assert typed `wrong_network`, assert no `eth_sendTransaction` in the RPC log, assert `transactionSubmitted=false`	**ALL PASS, only `eth_chainId` seen, zero funds spent**
Task 10 — complete lifecycle	Final integration verification	Amoy + Pinata + Supabase	`CERT-2026-273168`	issuance → 42 read-only checks (A/B/C, tamper, reconciliation, anchored rows, fail-open) → revocation → wrong-network re-run	issuance tx `0x823d7bc4…7803` block #50315729 status 1, 9328 ms total; **42/42 checks PASS**; reconciliation **24/24 anchored**; fingerprint-compat **24/24 + 24/24**; revoke tx `0xfd8fa7f8…` block #50321993 status 1 → A `revoked`, B `revoked`, C `match` + status revoked, Supabase updated post-receipt with hash unchanged

Testnet spend across the project: Task 3 = 10 issuance transactions; Task 4 = 1 issuance + 1 self-transfer; Task 10 = 1 issuance + 1 revocation + 0 wrong-network transactions. Wallet balance after Task 10: ~50.155 POL. Live test certificate `CERT-2026-273168` is **REVOKED** on-chain. [EM]

Evidence caveat: all live-experiment outcomes exist as **prose in `context_md/*.md`, not as machine-readable artifacts** — the live scripts print to console and write no files, and the benchmark run logs were deleted during Task 6 cleanup. Only the three `benchmarks/*.json` files are persistent artifacts. [OL]

---

22. Testing Evidence

22.1 FRAMEWORK AND COMMAND

Bun test (`bun:test`), command `bun test` / `bun run test` . `bunfig.toml` has no `[test]` section (only `[install]` supply-chain settings: `saveTextLockfile` , `minimumReleaseAge = 86400`). No CI (`.github/` absent). No other test framework present. `tsconfig.json` typechecks only `src/**` ; `test/` and `scripts/` run directly under Bun.

22.2 COMPLETE TEST INVENTORY — 123 TESTS IN 7 FILES, 28 DESCRIBE BLOCKS, 0 `IT()` CALLS

File	Tests	Describes
`test/certificate-fingerprint.test.ts`	20	2
`test/ipfs-upload.test.ts`	23	4
`test/issuance-timing.test.ts`	13	6
`test/method-c-extraction.test.ts`	14	2
`test/supabase-integrity.test.ts`	14	2
`test/transaction-confirmation.test.ts`	29	9
`test/verification.test.ts`	10	3
Total	**123**	**28**

22.3 WHAT EACH GROUP COVERS

Group	Coverage
certificate-fingerprint (20)	Canonical string and field order; key-insertion-order independence; trim determinism; internal whitespace significance; zero-padded ISO dates; TIMESTAMPTZ → date reduction and round-trip; verbatim non-ISO dates; determinism; protocol vector vs an independent WebCrypto oracle; **legacy formula equivalence**; `0x` +64 lowercase format; single-field mutations (recipient/course/date/ID); single-character mutation; UTF-8 Unicode; whitespace-equivalence; `2026-5-18` vs `2026-05-18`
ipfs-upload (23)	JWT resolution (absent/empty/trimmed); 11 validation cases (non-FormData, missing entry, string entry, empty, oversized, disallowed MIME, missing MIME, extension/MIME mismatch, valid PDF, PNG/JPEG, extensionless PDF); 5 Pinata HTTP behaviours (Bearer header + CID return, HTTP failure sanitisation, non-JSON body, missing `IpfsHash`, network failure); 4 end-to-end cases (no credential leaks no secret; invalid file never contacts Pinata; valid upload uses the server credential; Pinata 401 does not leak the JWT)
issuance-timing (13)	Durations non-negative and finite; total ≥ sum of sequential stages; double-start / end-unstarted throw; negative duration rejected; `success` refuses to finalize with an open stage; `measureStage` passthrough and throw propagation; failure records; success records; non-blocking stage tracking; PII-free structured logging (asserts no `recipientEmail
method-c-extraction (14)	Full round-trip on real generated PDFs (incl. embedded image); 4 tamper cases each changing extraction and fingerprint; 2 missing-field cases; malformed bytes; empty file; partial extraction; PDF string escaping; ISO-datetime normalisation
supabase-integrity (14)	7 consistency cases (untampered match, case-insensitivity, tampered name/course/date, re-keyed row, foreign fingerprint); 7 reconciliation cases (keep+override, hash-mismatch drop, case-insensitive compare, metadata-only drop, RPC-error fail-open, mixed batch, empty input)
transaction-confirmation (29)	See Section 15.6
verification (10)	4 input-validation cases (empty/short/non-hex → `invalid_hash` with no network; `isHashLike` boundary); 3 Method B QR equivalence cases; 3 Method C offline early returns (text-as-PDF, empty file, `documentByteHash` domain separation)

22.4 HERMETICITY

All 123 tests are offline. `fetch` is mocked; chain and Supabase lookups are injected stubs; timer behaviour uses a recording schedule seam; extraction and fingerprint recomputation touch no network. The only env interaction is `test/ipfs-upload.test.ts` saving/restoring `process.env.PINATA_JWT` itself (it needs no pre-existing value). Note that the import chain reaches `src/lib/supabase.ts`, which calls `createClient(...)` at module load, so Bun's auto-loaded `.env` can affect module initialisation even though no tested path performs a network call. [IV] [OL]

22.5 GATES RECORDED PER TASK

Task	`bun test`	`tsc --noEmit`	`vite build`
1 (fingerprint)	20 pass / 0 fail	exit 0	exit 0
2 (Pinata)	41 pass / 0 fail	0 errors	success
3 (issuance)	54 pass / 0 fail	0 errors	success
4 (confirmation)	83 pass / 0 fail	exit 0	exit 0
5 (IPFS)	85 pass / 0 fail	clean	passing
6 (verification)	95 pass / 0 fail	OK	OK
7 (Method C)	109 pass / 0 fail	OK	OK
8 (security)	109 pass / 0 fail	OK	OK
9 (Supabase)	123 pass / 0 fail	PASS (exit 0)	PASS (~12.7 s)
10 (final)	123 pass / 0 fail (×2 clean runs)	OK	OK

22.6 FLAKY RUNS — ACCURATE CHARACTERISATION

`context_md/2026-10-16_132041.md:71-73` and `2026-10-16_152855.md:49-50` record that **one** full-suite run exited 1 with **0 failures reported**, caused by an intermittent network rejection inside the `ipfs-upload` tests; subsequent full runs exited 0. This is a harness/network flake, not a failing assertion. It should be described exactly that way and not counted as a failed test campaign. [OL]

22.7 TYPECHECK / LINT / BUILD

- Typecheck: **`bunx tsc --noEmit`** (manually invoked; `tsconfig.json` has `noEmit: true` and includes only `src/**`). **No `typecheck` script exists in `package.json`.** [OL]
- Lint: `eslint .` with `@eslint/js` + `typescript-eslint` recommended + Prettier + react-hooks/react-refresh, plus one custom `no-restricted-imports` rule banning Next's `server-only` in favour of `@tanstack/react-start/server-only`. `@typescript-eslint/no-unused-vars` is **off**. **No security plugin.** [OL]
- Known environmental lint condition: `core.autocrlf=true` (CRLF checkout) vs Prettier's default `endOfLine: lf` produces ~2,741 repo-wide "Delete `CR`" errors; the repository-wide lint baseline is therefore broken for environmental reasons and must not be reformatted wholesale (`context_md/2026-10-14_204722.md:72-76`). [OL]
- Build: `vite build` (production) and `vite build --mode development`; preset `cloudflare-module`; Nitro output in `.output/`. [IV]

22.8 SECURITY SCANNING

Present: client-bundle static scans in Task 8/9/10 (patterns `PINATA_JWT`, `DEPLOYER_PRIVATE_KEY`, `service_role`, `pinata.cloud`, `PRIVATE_KEY`, `eyJhbGciOi`) with 0 findings; secret-leak assertions in tests; `bunfig.toml` `minimumReleaseAge = 86400` supply-chain guard; `context_md/` and `.env` gitignored. **Absent:** `npm audit` / `bun audit`, `eslint-plugin-security`, any CI, and any smart-contract static analysis (Slither was proposed in `context_md/2026-10-14_171533.md:102` but not run). [OL] [EXT]

---

23. Final System Properties

Property	Verdict	Why
Tamper evidence (metadata)	**PROVEN**	14 offline extraction tests + 4 fingerprint mutation tests + live 5/5 Method C tamper cases; scope limited to the four metadata fields in text PDFs
Blockchain anchoring	**PROVEN**	contract deployed; live issuance receipts with `status 1` and `blockNumber > 0`; fingerprint-compat 24/24 chain MATCH
Decentralized certificate proof	**PARTIALLY PROVEN**	the anchor is decentralized (public chain), but the app depends on a free public RPC, Pinata, and Supabase — a centralized dependency stack
Issuer authorization	**PARTIALLY PROVEN**	enforced by contract `require` and displayed by `isAuthorizedIssuer`; **no automated contract test**; only the deployer's authorization was exercised live
Revocation	**PROVEN**	one real revocation transaction (block #50321993, status 1); A → `revoked`, B → `revoked`, C → `match` + status revoked; Supabase updated post-receipt with hash unchanged; irreversible in code
QR verification	**PROVEN (transport only)**	payload build/extract equivalence tested; Method B consistent with A 10/10; **no in-app scanning**
Document metadata verification	**PROVEN within scope**	10/10 authentic `match`, live tamper detection; scope = CertiProof-generated text PDFs
IPFS persistence	**PARTIALLY PROVEN**	36/36 byte-identical retrievals and one delayed recheck over ~10.9 minutes; that is a short horizon and a single vendor
Cryptographic integrity	**PROVEN**	SHA-256 WebCrypto, protocol vector test, domain separation tests, fingerprint consistency 23/23
Pending transaction safety	**PROVEN**	29 unit tests + a live pending→confirmed run at block 50247727
Wrong-network protection	**PROVEN (script-level)**	live check with zero funds spent; **not covered by an automated unit test**
Supabase reconciliation	**PROVEN (app-side)**	14 tests + live 24/24 anchored reconciliation; **database-level enforcement is not applied [EXT]**
Chain-authoritative verification	**PROVEN**	every status path reads the contract tuple; metadata requires fingerprint agreement
Confidentiality of documents	**NOT PROVEN / NOT PROVIDED**	IPFS content is public by design; the audit warns public IPFS content must be treated as potentially accessible
Academic legitimacy of content	**NOT PROVIDED (by design)**	`context_md/audit.md:228-230`
Scalability	**NOT PROVEN**	linear O(n) scan; ~6 s mean at n≈28; no larger-scale experiment performed
Production readiness	**NOT PROVEN**	no auth, no rate limiting, no RLS, no CI, no contract tests, testnet only
Mainnet behaviour / cost	**NOT MEASURED**	testnet only; gas figures stale and non-persisted

24. Limitations

Rigorous, non-hidden. Each is tagged with its evidence tier.

#	Limitation	Detail	Tier
L1	**Supabase RLS not applied or verified**	`supabase/rls-hardening.sql` is an explicitly labelled proposal; the repo contains no live schema, no migrations, no applied-state evidence. A hostile client can insert format-valid rows and flip `status` at the database layer.	[EXT] [OL]
L2	**Unauthenticated IPFS upload**	The single server function is publicly callable with no authentication and **no rate limiting anywhere in the repo**. Risk is Pinata quota/cost abuse. Documented in Task 8 as requiring edge/CDN rate limiting plus Pinata quota alerts.	[OL] [EXT]
L3	**Public RPC dependency**	Verification and benchmarks depend on `polygon-amoy-bor-rpc.publicnode.com`, shared infrastructure that throttles bursts and caps `eth_getLogs` at a 10,000-block range.	[OL]
L4	**`getLogs` range limitation**	`fromBlock: 0` scans (~50.3M blocks) always fail on the free RPC; measured `blockchainRecordPass: 0/10`. Mitigated on 2026-10-16 by a receipt-verified candidate fallback, but event-scan recovery still needs an archive/bounded-range RPC.	[EM] [OL]
L5	**Method C PDF scope**	Text PDFs generated by the app's jsPDF layout only. Label-anchored parsing is layout-dependent; other generators, other fonts, or other layouts are unsupported.	[OL]
L6	**No OCR**	Scanned or image-only PDFs yield all-null extraction and `complete = false`.	[OL]
L7	**Standard-font PDF extraction constraints**	Extraction depends on jsPDF emitting unencrypted literal strings `(…)` in the content stream; compressed/encoded streams or embedded-font encodings are not handled.	[OL]
L8	**Metadata/display dependency**	Method C's registry comparison requires a Supabase row; chain-only certificates return `not_found` with a "metadata unavailable" marker rather than a full comparison.	[OL]
L9	**Session-scoped pending registry**	`pending-transactions.ts` is an in-memory `Map`; a page reload loses the payload. Pending records are never written to Supabase.	[OL]
L10	**One-shot pending confirmation**	`getTransactionState` performs exactly one RPC lookup with no polling; a user must manually re-trigger a check.	[OL]
L11	**Chain-only metadata is empty by design**	Method A on a chain-only certificate shows empty recipient/course and a metadata check of `absent`.	[OL]
L12	**Performance variance**	IPFS upload stdDev 12.0 s against a 9.5 s mean; Method A stdDev 1.68 s against a 6.0 s mean. Small-n (n=5 per size; n=1 for the current issuance baseline).	[EM] [OL]
L13	**Testnet environment**	All results are Polygon Amoy testnet results. Testnet success is not production readiness.	[OL]
L14	**Small sample sizes**	Verification n=10/method; IPFS n=5/size; **issuance n=1**; tamper live cases n=1 each; revocation live cases n=1.	[EM] [OL]
L15	**No smart-contract test suite**	`test/CertChainRegistry.test.ts` was planned and never written; contract behaviour is verified by inspection and one live revocation only.	[OL]
L16	**No CI**	No `.github/`; gates are run manually per session.	[OL]
L17	**No `typecheck` script / broken repo-wide lint baseline**	`tsc --noEmit` is invoked by hand; Prettier vs CRLF produces ~2,741 environmental lint errors.	[OL]
L18	**Frontend has no auth guards**	`dashboard.tsx` is a layout shell; login checks `checkIssuerAuth` but is advisory. Enforcement lives only in the contract.	[OL]
L19	**Dashboard "Verified" metric is not verification events**	`metrics.verified = certificates.filter(c => c.chain.transactionHash).length` — it counts records holding a transaction hash.	[OL]

#	Limitation	Detail	Tier
L20	**Single immutable `superAdmin`**	No transfer/renounce, no pause, no upgradability; admin key loss or compromise is unrecoverable.	[OL]
L21	**Existence keyed on `ipfsCID` length**	A registration with an empty CID would be written yet invisible to `getCertificate` / `certificateExists`.	[OL]
L22	**Unbounded enumeration + O(n) verification**	No pagination on `certificateIds`; the frontend loops all ids per verification.	[OL]
L23	**Dead code present**	`src/lib/certiproof.ts` (mock layer, contains the only `documentByteHash` / `cert.hash` conflation) and most of `src/lib/blockchain/verify.ts` are unimported.	[OL]
L24	**Stale documentation in-repo**	`PROJECT_SUMMARY.md` still describes the mock era (“No tests written”, “No smart contract written”, “0%” everywhere) and a different contract address.	[OL]
L25	**Superseded / stale measurement artifacts**	`issuance-results.json` was overwritten to n=1; `verification-results.json` predates both the Task 7 extractor fix and the Task 10 event-scan fix, so its tamper and blockchain-record figures describe pre-fix behaviour.	[OL]
L26	**Live results are not machine artifacts**	Live scripts print to console and write nothing; their outcomes exist only as prose in `context_md/`. Run logs were deleted.	[OL]
L27	**No host/machine metadata recorded**	No CPU/RAM/OS/runtime versions in any benchmark JSON; no commit hash per run.	[OL]
L28	**No warm-up phase in any harness**	Any warm-cache effect on run 1 is unmeasured.	[OL]
L29	**No dependency advisory audit performed locally**	Task 8 records it as out of local scope.	[EXT]
L30	**No contract static analysis performed**	Slither was proposed, never run.	[EXT]
L31	**IPFS content is public**	Confidentiality is not provided; `context_md/audit.md:213-214` warns public IPFS content must be treated as potentially accessible.	[OL]
L32	**No email delivery, no settings persistence, no in-app PDF re-download of a QR-bearing PDF**	Settings are client state only; these were simulated in the mock era and are not claimed as functional.	[OL]

25. Future Work

Derived strictly from the limitations above.

A. ENGINEERING IMPROVEMENTS

1. Persist the pending-transaction registry (Supabase table or IndexedDB) so a reload does not lose in-flight work. (from L9) [FW]
2. Replace one-shot confirmation with bounded polling or an event-driven/subscription-based confirmation watcher. (L10) [FW]
3. Add a `typecheck` script and a CI workflow running `bun test`, `tsc --noEmit`, `eslint`, `vite build`, and the client-bundle secret scan. (L16, L17) [FW]
4. Delete or clearly quarantine dead code (`certiproof.ts`, unused `verify.ts` functions). (L23) [FW]
5. Refresh or remove stale in-repo documentation (`PROJECT_SUMMARY.md`) and correct the `demo-data.ts` contract address. (L24, Section 35) [FW]
6. Re-run and persist the issuance benchmark with n ≥ 10 under the current confirmation implementation. (L25) [FW]
7. Record host/runtime metadata and a commit hash in every benchmark JSON. (L27) [FW]

8. Add a warm-up phase to each harness. (L28) [FW]

B. SECURITY IMPROVEMENTS

9. Apply `supabase/rls-hardening.sql` in a staging project, run its preflight queries, and verify the live schema. (L1) [EXT]
10. Move issuance persistence and status flips server-side (edge function + service-role key) so the browser can never write. (L1) [FW]
11. Authenticate and rate-limit the IPFS upload server function (edge/CDN rate limiting, per-issuer quotas) and enable Pinata quota alerts. (L2) [EXT]
12. Rotate the Pinata JWT if it was ever present in a deployed bundle, and record that decision. [EXT]
13. Add a smart-contract test suite (authorization, duplicate registration, nonexistent ids, revocation, repeated revocation, events, boundaries) on a local chain. (L15) [FW]
14. Run contract static analysis (Slither) and a dependency advisory audit. (L29, L30) [EXT]
15. Add automated tests for the wrong-network guard and for the issuance/revoke paths. (Section 16 gap) [FW]
16. Add frontend route guards as UX controls, documented as non-authoritative. (L18) [FW]
17. Rename or annotate the dashboard “Verified” metric to reflect what it counts. (L19) [FW]

C. RESEARCH EXTENSIONS

18. OCR-based extraction to extend Method C to scanned documents. (L6) [FW]
19. Support for additional PDF generators/fonts/encodings, with a stated extraction scope. (L5, L7) [FW]
20. A controlled tamper-detection study with a larger, systematically generated tamper corpus (multiple fields × multiple edit types × multiple document generators) to replace the current n=1-per-case live evidence. [FW]
21. A user/usability study with verifiers (employers) — no usability data exists. [FW]
22. A comparative evaluation against the closest prior systems (Bloom-filter-based verification, NFT credential schemes) on the same metrics. [FW]
23. Formal threat-model treatment and, optionally, a verification of the fingerprint protocol’s collision/canonicalisation properties.

D. SCALABILITY IMPROVEMENTS

24. Replace the linear scan with an on-chain mapping from fingerprint → id, or an off-chain index/event-derived index, in a **new contract version** (the deployed contract cannot be changed in place). (L22) [FW]
25. Benchmark verification latency as a function of registry size to convert the O(n) code observation into a measured claim. (L22) [FW]
26. Use a dedicated/archive RPC provider with bounded `eth_getLogs` ranges so registration-event recovery works without the receipt fallback. (L3, L4) [EXT] [FW]

E. DEPLOYMENT IMPROVEMENTS

27. Mainnet or a production-grade Polygon deployment with a documented cost analysis. (L13) [FW]
28. Persisted hosting (the build preset is `cloudflare-module` but no `wrangler.toml` /deployment config is committed). [FW]
29. Email delivery of certificates and settings persistence, or explicit removal of those UI affordances. (L32) [FW]
30. Multi-institution administration (issuer management UI) rather than script-based `add-issuer.cjs`. (L20) [FW]

---

26. IEEE Paper Contributions

Ranked strongest to weakest, with an explicit honesty note.

C1 (STRONGEST) — A CHAIN-AUTHORITATIVE VERIFICATION ARCHITECTURE THAT REMAINS CORRECT AGAINST A MALICIOUS METADATA STORE

- **Statement:** certificate identity, issuer, status, and timestamp are read from the contract; database metadata is displayed only when its recomputed fingerprint equals the on-chain hash; rows that do not anchor are dropped; status is always overridden from the chain; writes are constrained by chain-derived identity.
- **Technical evidence:** `isSupabaseMetadataConsistentWithChain`, `reconcileSupabaseRowsWithChain`, `fetchVerifiedCertificateFromSupabase`, metadata gating in `verifyCertificate`, dual-key `updateCertificateStatusInSupabase`, chain-first `syncCertificatesFromChain`.
- **Experimental evidence:** 14 offline integrity tests covering tamper, re-keying, foreign-fingerprint, case-insensitivity, fail-open, and mixed-batch scenarios; live reconciliation 24/24 anchored.
- **Why it matters:** most blockchain-credential designs treat the database as trusted or add a chain read without defining the authority boundary. This makes the boundary explicit and enforceable in code.
- **Novel vs existing work: REQUIRES LITERATURE VERIFICATION.** The repository has no completed comparison table.

C2 — A CANONICAL METADATA FINGERPRINT PROTOCOL WITH STRICT HASH-DOMAIN SEPARATION, PROVEN BACKWARD COMPATIBLE

- **Statement:** identity = SHA-256 over a fixed-order, separator-joined, trimmed, date-normalised canonical string, encoded `0x`+64 lowercase hex; the document byte hash is a strictly separate domain that is never compared to the registry key.
- **Technical evidence:** `certificate-fingerprint.ts`; 20 protocol tests including a protocol vector against an independent WebCrypto oracle and legacy-formula equivalence.
- **Experimental evidence:** `check-fingerprint-compat.ts` → 24/24 stored MATCH and 24/24 chain MATCH; fingerprint consistency 23/23 across recomputed/chain/Supabase.
- **Why it matters:** it resolves a real, measured failure mode (byte-hash vs metadata-hash mismatch made every legitimate document verification fail) **without breaking existing on-chain records**.
- **Novelty:** the protocol itself is not algorithmically novel (SHA-256 over a canonical string) — **`context_md/2026-10-14_171533.md:79` records "NO algorithmic novelty."** The contribution is the domain separation and the compatibility proof.

C3 — DOCUMENT RE-DERIVATION: VERIFYING A PRESENTED PDF BY RECOMPUTING THE REGISTRY FINGERPRINT FROM ITS OWN PRINTED FIELDS

- **Statement:** a presented certificate PDF is parsed with label-anchored extraction; the extracted fields are re-hashed with the same canonical protocol; `match` requires both the registry-metadata fingerprint and the extracted-document fingerprint to equal the on-chain anchor **and** all individual fields to agree.
- **Technical evidence:** `extract-certificate-metadata.ts` with escape handling, static-label rejection, explicit nulls, `complete` semantics; 14 extraction tests.
- **Experimental evidence:** 10/10 authentic `match`; pre-fix 2/4 tamper detection → post-fix live 5/5.
- **Why it matters:** it binds a physical document to an on-chain identity without embedding a trusted field, and it demonstrated a concrete detection gap (recipient/course) that was measured before and after.
- **Novelty: REQUIRES LITERATURE VERIFICATION.**

C4 — HONEST TRANSACTION OUTCOME SEMANTICS (PENDING AS A FIRST-CLASS NON-SUCCESS STATE)

- **Statement:** only a real mined receipt with `status === 1` **and** `blockNumber > 0` counts as success; timeout yields `pending`; synthetic receipts are structurally rejected; replacement/cancellation handled; timers always cleaned up.
- **Technical evidence:** `confirmation.ts` + 29 tests.
- **Experimental evidence:** the pre-fix defect was *measured* — 5/10 issuance runs reported success while the transaction mined 37–166 s later; the fixed machine was then validated live (pending → confirmed at block 50247727).
- **Why it matters:** this is an empirical, reproducible demonstration of a failure class that a paper can generalise to blockchain application design.
- **Novelty: REQUIRES LITERATURE VERIFICATION; likely the most generalisable engineering finding in the project.**

C5 — PRE-SIGNING NETWORK VALIDATION PREVENTING NO-OP "SUCCESSFUL" TRANSACTIONS

- **Statement:** a chain-id guard runs before any fingerprint/PDF/IPFS/signing side effect; signing on a non-target chain can mine a status-1 no-op that a receipt-validating machine would otherwise classify as success.
- **Experimental evidence:** live check showing only `eth_chainId` was ever called and zero funds were spent.
- **Novelty: REQUIRES LITERATURE VERIFICATION.**

C6 (WEAKEST) — REPRODUCIBLE MEASUREMENT OF AN INTEGRATED ISSUANCE/VERIFICATION PIPELINE ON A PUBLIC TESTNET

- **Statement:** three benchmark harnesses drive the real application code and persist raw results: IPFS 39/39 uploads with 36/36 byte-identical retrievals and content-addressed dedup; verification 10/10 across three methods with negatives; issuance timings.
- **Weakness:** small n; issuance baseline reduced to n=1 by overwrite; no host metadata; no commit hash; live results not persisted as artifacts.

CLAIMS THAT MUST BE CITATION-SUPPORTED IN AN IEEE PAPER

1. Any statement about the state of academic credential fraud (prevalence, cost) — **no such statistic exists in this repository**; the only such figures found came from a reference paper (~2M counterfeit degrees, ~$600B/yr, 10% fraudulent applications, `context_md/2026-10-14_161651.md:39-40`) and must be cited to that source, not asserted.
2. Any novelty claim relative to prior blockchain-credential systems.
3. Any claim that blockchain anchoring improves verification over existing practice.
4. Any claim about SHA-256 security properties.
5. Any claim about IPFS permanence, deduplication, or content addressing as general properties (only 36 cycles were measured here).
6. Any comparison to Bloom-filter, NFT/ERC-721, or verifiable-credential approaches.
7. Any claim about gas cost, since the recorded numbers are stale and non-persisted.

---

27. Safe Claims vs Unsupported Claims

A. CLAIMS STRONGLY SUPPORTED BY IMPLEMENTATION + EXPERIMENTS

1. Certificates are registered on Polygon Amoy in contract `CertChainRegistry` at `0x0992…DF50`, with real transactions whose receipts report `status 1` and non-zero block numbers. [EM]
2. The certificate identity is SHA-256 over a canonical, fixed-order, separator-joined, trimmed, date-normalised four-field string, output as `0x` + 64 lowercase hex. [IV] [EM]
3. That protocol is byte-identical to the legacy issuance formula, so all existing records retain their hashes (24/24 stored and 24/24 chain MATCH). [EM]
4. The document byte hash and the metadata fingerprint are separate domains and are never compared in production code. [IV]
5. Method A returns `verified` / `revoked` / `not_found` / `error` with status read from the chain, and 10/10 valid certificates verified in the benchmark. [EM]
6. Method B's QR payload is `https://certiproof.app/verify/<fingerprint>` and behaves identically to Method A (10/10 consistency). [EM]
7. Method C re-extracts recipient, course, issue date, and certificate ID from a CertiProof-generated PDF and requires the recomputed fingerprint to equal the on-chain hash (10/10 match). [EM]
8. Recipient/course/date/ID tampering in such PDFs changes the recomputed fingerprint; measured live as 5/5 detected. [EM]
9. Malformed hashes are rejected offline before any network call (4/4), and unknown hashes return `not_found` (5/5). [EM]
10. Timeout during confirmation yields `pending`, never success; a `blockNumber: 0` receipt is never accepted; validated by 29 tests and a live pending→confirmed run. [EM]
11. Issuing or revoking on a non-Amoy chain throws a typed `wrong_network` error with no transaction submitted and no funds spent. [EM]

12. Pinata credentials are read only in a server-only module; client-bundle secret scans found no credential material. [EM]
13. IPFS uploads succeeded 39/39 and retrievals were byte-identical 36/36, with content-addressed deduplication observed (same file → same CID). [EM]
14. Database metadata is only displayed when its recomputed fingerprint matches the on-chain hash, and non-anchoring rows are dropped from the dashboard. [IV] [EM]
15. The application never deletes Supabase rows and only updates `status`, constrained by the chain-derived hash. [IV]

B. CLAIMS THAT REQUIRE CITATIONS OR LITERATURE COMPARISON

- Any novelty or “first” claim. [The repo’s own audit records “NO algorithmic novelty”.]
- Any claim that CertiProof outperforms or is more secure than existing systems.
- Any statement about the prevalence or cost of credential fraud.
- Any general claim about IPFS reliability or permanence beyond the measured 36 cycles.
- Any claim about gas cost or economic advantage.
- Any claim about scalability beyond the measured n≈28 registry.
- Any claim about user acceptance, usability, or adoption.

C. CLAIMS WE MUST NOT MAKE (WITH THE EXACT OVERCLAIM FLAGGED)

Dangerous claim	Why it is false or unsupported here	Where a similar overclaim already exists
”100% secure” / “zero vulnerabilities”	RLS unapplied, unauthenticated upload, no rate limiting, no contract tests, no static analysis	—
”Completely decentralized”	Depends on one public RPC, one pinning vendor, one database	—
”Impossible to forge”	Only the four metadata fields are checked; no OCR; no visual-tamper detection; a forged identity that is never registered is simply `not_found`	—
”100% reliable” / “100% reliable IPFS”	36 cycles over one day, one gateway; latency variance up to 54 s	The stored rate strings `39/39 (100%)` must be reported **with n and scope**, never as a general guarantee
”Production ready”	No auth, no CI, no RLS, testnet only	—
”Better than all existing systems”	No comparative evaluation performed	—
”Tamper detection rate 100%” without qualification	The stored benchmark says 2/4 (50%); the later live run says 5/5 with n=1 per case	Both numbers exist — quote both with dates
”The blockchain proves the certificate is genuine”	It proves registration, issuer, status, and integrity of the metadata; not academic truth	`context_md/audit.md:228-230` forbids this
”Verification takes ~X seconds” as a system property	~6 s mean is dominated by a linear scan at n≈28 on a free RPC	—
”No tests”	123 tests exist	`PROJECT_SUMMARY.md:152,173` still claims 0 tests — **stale, must not be cited**

28. Required Figures

Specification only — no figure has been generated.

FIG. 1 — SYSTEM ARCHITECTURE

- **Purpose:** establish the components and the trust boundary in one view.
- **Components:** Issuer browser (React/TanStack Start app, MetaMask); server-only IPFS function; Pinata; Polygon Amoy `CertChainRegistry`; read-only RPC provider; Supabase; Verifier browser; QR code.
- **Data flow:** issuance path (form → fingerprint → PDF → IPFS → contract → Supabase) and verification path (hash/QR/PDF → read-only RPC + gated Supabase → result).
- **Supports:** the authority hierarchy claim (C1) and the trust-boundary description in Section 5.

FIG. 2 — CERTIFICATE ISSUANCE WORKFLOW

- **Purpose:** show the ordered stages and the point at which each side effect occurs.
- **Components:** the seven instrumented stages plus the pre-guard, with branch arrows for wrong-network, IPFS failure, submission error, revert, and timeout→pending.
- **Data flow:** inputs → canonical string → fingerprint → PDF blob → CID → tx hash → receipt → certificate → Supabase row.
- **Supports:** the claim that Supabase is written only after a real receipt; the wrong-network side-effect prevention claim (C5).

FIG. 3 — VERIFICATION WORKFLOW A / B / C

- **Purpose:** show three entry points converging on one chain-authoritative decision.
- **Components:** hash input and QR extraction feeding `verifyCertificate`; PDF upload feeding `analyzeDocument`; shared chain lookup; the 7 Method A checks; Method C's two fingerprint comparisons.
- **Supports:** the verification-correctness claims (Section 20.5) and the C3 contribution.

FIG. 4 — BLOCKCHAIN / IPFS / SUPABASE DATA MODEL

- **Purpose:** make the authority split concrete.
- **Components:** contract struct (6 fields + mapping key + array), IPFS object (bytes ↔ CID), Supabase table (12 columns), with each field labelled authoritative / untrusted-display / content-addressed.
- **Supports:** the "Supabase cannot manufacture a verified certificate" claim (C1).

FIG. 5 — FINGERPRINT GENERATION AND VERIFICATION

- **Purpose:** specify the protocol unambiguously.
- **Components:** the four fields in fixed order → trim/normalize → `|` join → UTF-8 → SHA-256 → `0x` +64 hex; plus a parallel branch showing document byte hash as a **separate, never-compared** domain.
- **Supports:** contributions C2 and the domain-separation claim (Section 7).

FIG. 6 — TRANSACTION CONFIRMATION STATE MACHINE

- **Purpose:** show `confirmed` / `failed` / `pending` with every reason code and the receipt-validity gate.
- **Components:** `isValidReceipt` (blockNumber > 0), `classifyReceipt` (status 1/0/null), timeout → final check → `timeout_no_receipt`, wait-error → `wait_error_no_receipt`, `TRANSACTION_REPLACED` cancelled/repriced branches, submission error.
- **Supports:** contribution C4 and the measured pre-fix defect.

FIG. 7 — TAMPER DETECTION WORKFLOW (METHOD C)

- **Purpose:** contrast the pre-fix and post-fix pipelines.
- **Components:** PDF → literal-string collection → unescape → sanitize → label-anchored field reads (with static-label rejection) → `complete` flag → recomputed fingerprint vs chain anchor; plus the pre-fix branch where recipient/course were placeholders and could never differ.
- **Supports:** contribution C3 and the 2/4 → 5/5 detection change.

FIG. 8 — PERFORMANCE RESULTS

- **Purpose:** present measured latency honestly.

- **Components:** (a) Method A stage breakdown showing `searchAndDecision` ≈ 99.9% of total; (b) Method C stage breakdown; (c) IPFS upload and retrieval latency by file size with error bars (min/max or ±stdDev); (d) a clearly labelled n=1 issuance stage waterfall.
- **Supports:** the dominant-latency claim and the limitations L12/L14/L22.

Optional Fig. 9 — Registry growth timeline, if the certificate-count discrepancy (Section 35) is resolved with dated evidence.

---

29. Required Tables

#	Title	Columns	Data source	Purpose
T1	Existing system vs proposed system	Aspect, Conventional certificate process, CertiProof	Sections 1.10, 2.2, `audit.md`	Motivation; must stay qualitative unless a citation supplies numbers
T2	Smart contract interface	Function, Visibility, Modifier, Revert strings, Event	`contracts/CertChainRegistry.sol`	Reproducibility of the on-chain layer
T3	On-chain vs off-chain data placement	Data item, Chain, IPFS, Supabase, In-memory	Section 6.2	Trust-boundary argument
T4	Verification method comparison	Method, Input, Process, Sources, Detects, Limitations	Section 13.2	Core methodological table
T5	Security threat analysis	Threat, Attack, Defense, Result, Remaining risk	Section 18	Security analysis
T6	Tamper detection results	Case, Field, Pre-fix outcome, Post-fix outcome, Evidence	`negatives.tamper[]` + `live-method-c-check`	The clearest correctness result — **must show both pre- and post-fix**
T7	Performance evaluation	Operation, n, min, median, mean, max, stdDev	three benchmark JSONs	Results; n and units mandatory in the caption
T8	Verification outcome rates	Measure, Result, n	`reliability.*` blocks	Correctness summary
T9	Test coverage / testing evidence	File, Tests, What it covers, Hermetic?	Section 22.2–22.4	Reproducibility
T10	Limitations and future work	ID, Limitation, Tier, Mitigation	Sections 24–25	Honest closing analysis
T11	Fingerprint protocol specification	Rule, Exact behaviour	Section 7.1	Allows independent reimplementation
T12	Environment and configuration	Parameter, Value, Recorded where	Section 19.1	Experimental setup

---

30. Literature Review Requirements

No citations are invented here. The repository contains ten reference PDFs whose bibliographic metadata was extracted into `context_md/2026-10-14_171533.md:45-60`; those extractions are second-hand and **each must be re-verified against the publisher record before inclusion**.

Research area	Type of papers needed	What CertiProof should compare against	Gap to investigate
Blockchain-based academic credential verification	Peer-reviewed system papers with an implemented evaluation	Registry design, anchoring choice, verification path	Whether an explicit chain-vs-DB authority hierarchy is already treated
Digital / verifiable credentials & decentralized identity	Standards (W3C VC/DID) and system papers	CertiProof's non-W3C, fingerprint-based model	Whether standards-based approaches subsume the fingerprint protocol
Academic credential fraud	Empirical/survey studies with statistics	Any prevalence/cost claim in the paper	Must supply the numbers the repo lacks
Blockchain + IPFS for document integrity	Implementation papers with measured results	IPFS reliability and content addressing claims	Whether content-addressed retrieval integrity is already established
Cryptographic hash-based document integrity	Foundational + applied papers	The metadata fingerprint vs byte-hash distinction	Whether canonical-metadata hashing for documents is established practice
QR-based certificate verification	System papers	The QR transport path	Whether QR verification is treated as a contribution elsewhere
Smart-contract-based credential systems	Contract-level analyses	Contract design, access control, revocation	Whether single-admin, non-upgradeable registries are critiqued as a pattern
Certificate revocation mechanisms	Design papers	On-chain status flag vs CRL/OCSP-style designs	Comparative revocation analysis
Blockchain scalability / registry lookup	Benchmark papers	The O(n) linear scan and its measured latency	Whether index-free registries are known-limiting
Security analysis of blockchain applications	Threat-model methodologies	The Section 18 threat model	Whether a standard methodology should be adopted instead of an ad-hoc list

Reference papers already identified in the project (metadata from `context_md`, unverified): Cardenas-Quispe & Pacheco 2025 (*Sci. Rep.*, DOI 10.1038/s41598-025-93913-6); ShikkhaChain 2025 (arXiv 2508.05334v2); de Alwis, Shrestha & Sarker 2025 (*Discover Education* 4:57, DOI 10.1007/s44217-025-00449-y); Pondkule & Kothari 2025 (*IJSSIS* 18(1), DOI 10.2478/ijssis-2025-0001 — near-identical stack: IPFS + SHA-256 + Ethereum + Polygon L2, conceptual, not empirically evaluated); Gafforova et al. 2026 (*JISIS* 16(2), DOI 10.58346/JISIS.2026.I2.005); Byun et al. 2025 (arXiv 2502.15653v1); Haque et al. 2025 "SALF" (*JASTT*, DOI 10.38094/jastt62295); Abu Bakar et al. 2025 (IOS Press FAIA, DOI 10.3233/FAIA250509); Bokari & Dewi 2025 (*MJM&F*); Priyadarshini et al. 2025 (*IEEE Access* 13:27037-27049, DOI 10.1109/ACCESS.2025.3539180 — noted as the closest competitor, Bloom-filter based).

Rules carried from the audit (`context_md/audit.md:183-198`): verify every citation's title, authors, venue, year, pages, and DOI against a publisher/index record; ensure every reference is cited and every citation resolves; compare against cited systems rather than asserting absolute novelty; distinguish implemented features from proposed/future work; never invent sources or DOIs.

---

31. 30-Second Explanation

CertiProof turns an academic certificate into something a stranger can check without calling the university. When a certificate is issued, four fields — ID, recipient, course, date — are canonicalised and hashed with SHA-256; that fingerprint goes onto the Polygon blockchain along with the certificate's IPFS pointer, the issuer's wallet, and a timestamp. To verify, anyone pastes the fingerprint, scans the QR code, or uploads the PDF; the app reads the blockchain directly, with no wallet and no fee, and says *verified*, *revoked*, or *not found*. The database that makes the page pretty is deliberately untrusted: if it disagrees with the chain, it is ignored. And the PDF itself can be checked, because the printed fields are re-read and re-hashed and must reproduce the fingerprint anchored on-chain.

32. 1-Minute Explanation

Academic certificates today are PDFs you can edit and a phone number you can call. CertiProof replaces both. An authorised institutional wallet issues a certificate through a web app: the app derives a SHA-256 **metadata fingerprint** from a canonical four-field string, renders the PDF with jsPDF, pins the PDF to IPFS through a server-side Pinata call so the credential never reaches the browser, and submits `registerCertificate` to a Solidity registry on Polygon Amoy. Only a real mined receipt — `status 1`, non-zero block number — counts as success; a timeout is reported as *pending*, never as success, and a signing attempt on the wrong network is refused before any money or storage is spent. Verification is free and wallet-free. Method A looks the fingerprint up in the registry and reports seven checks, with revocation status read from the chain, never from the database. Method B is the same thing through a QR-encoded URL. Method C uploads the PDF, extracts the printed recipient, course, date, and ID, and requires the recomputed fingerprint to equal the on-chain anchor — so an edited PDF no longer passes. The database is treated as hostile: any row whose metadata does not re-hash to the chain value is stripped or dropped, and status always comes from the chain. The result was measured on a real testnet: 10/10 authentic verifications across three methods, 5/5 tampered documents detected, 39/39 IPFS uploads with byte-identical retrievals, and 123 offline tests guarding the protocol.

33. 3-Minute Explanation

The problem. An academic certificate is a self-contained document with no external anchor. Its authenticity can only be checked by asking the institution, which does not scale, is slow, and cannot answer for institutions that are unreachable or uncooperative. Editing one printed name produces a file that no viewer can distinguish from a genuine one.

The design. CertiProof separates three concerns that are usually conflated. *Identity* is a SHA-256 fingerprint over a canonical string — fixed field order, `|` separator, trimmed fields, dates normalised to `YYYY-MM-DD` — encoded `0x` + 64 lowercase hex. This is deliberately a hash of the certificate's **semantic content**, not of the file bytes, because file bytes change whenever the renderer changes while the semantic content should stay stable. *Anchoring* writes that fingerprint, the IPFS CID, the issuer address, the certificate number, and a block timestamp into a deployed contract; revocation flips a status flag irreversibly and emits an event. *Presentation* — the Supabase row carrying names, courses, and transaction hashes — is explicitly untrusted: with a public anon key in the browser, any client can write, so every displayed row must re-hash to the chain value before its fields are shown, rows that do not anchor are dropped, status is always overridden from the chain, and a status update is refused unless the on-chain identity for that ID is positively found.

The verification story. Three entry points converge. Hash entry and QR scan run the same read-only pipeline against a public RPC — no wallet, no gas — reporting existence, issuer authorisation, revocation, confirmation depth, and whether displayed metadata re-derives to the anchor. Document upload is the strongest check: the PDF's literal text strings are collected and unescaped, four fields are read relative to fixed anchor labels with static-label rejection, and the extracted values are re-hashed with the identical protocol; `match` requires both the registry metadata and the extracted document to reproduce the on-chain hash, and every individual field to agree. Incomplete extraction can never pass, because a missing field produces an explicit null rather than a placeholder.

The engineering findings. Three measured defects deserve publication. First, the original forensic path hashed file bytes while issuance hashed canonical metadata, so *every* legitimate upload failed to verify — a live-confirmed mismatch that motivated the single canonical protocol, proven backward compatible at 24/24 stored and 24/24 chain comparisons. Second, the document extractor once returned a placeholder for recipient and course and reported `match` regardless, so a benchmark measured only 2 of 4 tamper cases detected; after the rewrite, a live run detected all 5 cases including recipient and course. Third, and most generalisable, the transaction path used to fall back to a synthetic receipt when the 60-second wait timed out, reporting **success** while the transaction was still pending — measured at 5 of 10 issuance runs, with those transactions actually mining 37 to 166 seconds later. The replacement state machine accepts only a real receipt with `status 1` and a non-zero block number, treats timeout as a first-class pending state, handles cancelled and repriced replacements, and is covered by 29 tests plus a live pending-to-confirmed run.

The measured reality. Verification averages about 6.0 seconds on a 28-certificate registry, of which roughly 99.9% is a linear on-chain scan — the system's clearest scalability limitation. Document verification averages 0.55 seconds. IPFS uploads succeeded 39/39 with retrievals byte-identical 36/36 over roughly eleven minutes, and identical files collapsed to identical CIDs. All of this is testnet, single-vendor, small-sample evidence and is presented as such.

The honest boundary. The blockchain proves that a certificate was registered by an authorised wallet, has not been revoked, and still corresponds to its metadata. It does not prove that the student earned the credential, that the institution is legitimate, or that the PDF's visual design is untampered. The system depends on a free public RPC, a single pinning vendor, and a database whose row-level security has been proposed but not applied.

34. IEEE-Style Technical Summary

Abstract (draft, built only from measured facts — verify every number before submission). Academic certificate verification remains dependent on the issuing institution and vulnerable to document forgery, because a certificate PDF carries no authenticated binding to any authority. We present CertiProof, a certificate issuance and verification platform in which the certificate's identity is a SHA-256 fingerprint computed over a canonical, fixed-order, separator-joined, date-normalised four-field metadata string, anchored on the Polygon Amoy blockchain together with the issuer address, an IPFS content identifier, and an issue timestamp. Document bytes are pinned to IPFS through a server-side interface that keeps the pinning credential out of the client bundle. Verification is wallet-free and offered by three converging paths: direct fingerprint lookup, QR-encoded verification URL, and document upload, in which the presented PDF's printed fields are extracted by label-anchored parsing and re-hashed with the same canonical protocol so that the recomputed fingerprint must equal the on-chain anchor. Auxiliary database metadata is treated as untrusted: rows are reconciled against the chain, non-anchoring rows are dropped, displayed fields are shown only when their recomputed fingerprint matches the chain value, and revocation status is always read from the contract. Transaction outcomes are reported only from mined receipts with a successful status and a non-zero block number, with timeout modelled as a first-class pending state. In a public-testnet evaluation, all three verification methods succeeded on 10 of 10 valid certificates, malformed inputs were rejected in all 4 cases, unknown certificates returned not-found in all 5 cases, metadata tampering in presented documents was detected in all 5 live cases after an extractor revision (from 2 of 4 in a pre-revision measurement), fingerprint consistency across recomputed, on-chain, and stored values held for 23 of 23 certificates, and IPFS uploads succeeded in 39 of 39 attempts with 36 of 36 retrievals byte-identical. Verification latency averaged 6.02 s for hash lookup — dominated by a linear registry scan — and 0.55 s for document verification. We further report the measured failure modes encountered, including a document-byte-hash versus metadata-fingerprint domain mismatch and a receipt-fabrication path that reported transaction success before mining, together with their remediations. Limitations including reliance on a public RPC, unauthenticated IPFS upload, unapplied row-level security, text-only document extraction, and testnet-scale evaluation are stated explicitly.

Keywords: blockchain; academic credentials; certificate verification; SHA-256; IPFS; smart contract; tamper detection; Polygon.

Technical architecture summary: React 19 / TypeScript / TanStack Start front end; Solidity 0.8.24 `CertChainRegistry` (11 functions, 4 events, single immutable super-admin, boolean issuer authorisation) deployed on Polygon Amoy (chain ID 80002); ethers v6 with a read-only JSON-RPC provider for verification and a MetaMask-backed signer for issuance; jsPDF generation and label-anchored literal-string extraction; WebCrypto SHA-256; Pinata pinning behind a TanStack server function with a server-only import barrier; Supabase `certificates` table accessed with a public anon key and reconciled against the chain.

Security summary: authority is hierarchical (contract > receipt/log evidence > recomputed fingerprint > database row > rendered text); metadata poisoning is neutralised by fingerprint gating, row reconciliation, status override, and chain-constrained writes; wrong-network signing is blocked before side effects; no client-side secret material was found in build-output scans; residual risks are unapplied RLS, unauthenticated/unrate-limited upload, a single immutable administrator, and the absence of contract-level automated tests.

Experimental summary: three benchmark harnesses drive the real application code against public infrastructure and persist raw results; verification n=10 per method with negatives and tamper probes; IPFS 7 sizes × 5 runs plus deduplication, delayed-retrieval, and error probes; issuance timings with an EIP-1193 signing shim; plus read-only live checks for the full lifecycle, revocation, wrong-network behaviour, and fingerprint compatibility. All numbers are testnet measurements with the sample sizes and dates recorded, and none should be generalised beyond that environment.

---

35. Final Consistency Audit

Cross-checked items and every discrepancy found. **No discrepancy was silently resolved.**

Item	Values found	Verdict
Certificate ID format	`CERT-YYYY-NNNNNN` in code (`/^CERT-\d{4}-\d{4,6}$/`), app-generated ids such as `CERT-2026-273168` , `CERT-2026-233459` , `CERT-2026-361028` ; the proposed RLS policy encodes the same regex	**CONSISTENT**
Contract name	`CertChainRegistry` in the contract, artifact, BUILD_PLAN, master spec, audit	**CONSISTENT**
Chain / network	Polygon Amoy, chain ID 80002 (`0x13882`) everywhere	**CONSISTENT**
Smart contract pragma	`^0.8.24` source; 0.8.24 configured	**CONSISTENT**
Terminology: fingerprint vs byte hash	Production code consistently separates `metadataFingerprint` and `documentByteHash`	**CONSISTENT in production**; conflated only in dead code `certiproof.ts:230,250`
IPFS/CID terminology	`cid` , `ipfsCID` , `IpfsHash` (Pinata field), `ipfs_cid` (DB column) — all refer to the same value	**CONSISTENT**
Supabase role	Comments and code treat it as untrusted display metadata	**CONSISTENT**
Methods A/B/C	Routes, functions, and benchmark meta descriptions agree	**CONSISTENT**
Verification A/B/C outcomes	`verified/revoked/not_found/error` , `match/mismatch/not_found`	**CONSISTENT**
Live transaction evidence	Issuance `0x823d7bc4…7803` block #50315729; revocation `0xfd8fa7f8…` block #50321993; pending→confirmed block 50247727	**CONSISTENT** (prose only)
Revocation evidence	Contract irreversible `status=1` ; live revoke verified by A/B/C	**CONSISTENT**

DISCREPANCIES (EXPLICIT)

D-01 — Two contract addresses. `deployment.json:2` , `scripts/check-contract.cjs:4` , `scripts/live-e2e-check.ts:46` , `scripts/eval-metrics.ts:16` , `scripts/check-fingerprint-compat.ts:22` , and both benchmark JSONs use **`0x09921D25cB513793e17082Cce0fB4F520f0CDF50`** . `src/lib/demo-data.ts:7` and `PROJECT_SUMMARY.md:28` use **`0x9C41ba7fD3a1E0Cd4E2a17b6E30fD2c9A1b47E30`** . *Assessment:* `0x0992…DF50` is the deployed, exercised address (it is the value in the deployment artifact and in every measurement). `0x9C41…E30` appears only in demo/legacy material. **The paper must use `0x0992…DF50` and `demo-data.ts` should be corrected** (not done here — documentation-only mandate).

D-02 — Three Polygon Amoy RPC URLs. `https://polygon-amoy.drpc.org` (deploy script only); `https://polygon-amoy-bor-rpc.publicnode.com` (hardhat default, add-issuer, check-contract, and the frontend read-only provider — this is the one all measurements used); `https://rpc-amoy.polygon.technology` (`demo-data.ts:8` , appears unused). *Assessment:* report **publicnode** as the measured RPC; note the deploy script differs.

D-03 — Certificate-count inconsistency across time. `audit.md:39-41` records a project snapshot of **9** on-chain records and a manuscript claim of **11 (7 active, 4 revoked)**. `context_md/2026-10-14_171533.md:30` measured **16 (12 active / 4 revoked)** with 12 Supabase rows. `benchmarks/verification-results.json` (2026-10-15) records **28 on-chain, 23 Supabase rows, 23 backed, 5 chain-only, 5 revoked**. `check-fingerprint-compat` on 2026-10-16 compared **24** rows. *Assessment:* these are different points in a growing registry, not a contradiction — but **any count in the paper must be dated and re-measured read-only immediately before submission.** `audit.md` C-02 requires exactly this.

D-04 — Stale in-repo documentation. `PROJECT_SUMMARY.md` states “No tests written (no test framework installed)” (line 152), “Testing 0% — No tests” (line 173), “No smart contract written” (line 144), “No real blockchain integration” (line 143), and gives the D-01 address. All are **false against the current tree** (123 tests, deployed contract, real ethers integration). *Assessment:* `PROJECT_SUMMARY.md` describes the **mock era** and must never be cited as current state. `BUILD_PLAN.md` planned `test/CertChainRegistry.test.ts` (absent) and `getTransactionHistory()` (absent), and named `hardhat.config.ts` / `scripts/deploy.ts` where the actual files are `.cjs` .

D-05 — Tamper-detection figures disagree across artifacts. `benchmarks/verification-results.json` (2026-10-15T17:36Z) records `tamperDetected: "2/4 (50%)"` with recipient and course returning `match`. `context_md/2026-10-15_235500.md` (later the same day) records a live 5/5 after the extractor rewrite. *Assessment:* the JSON is a **pre-fix measurement** and is stale; the live result is post-fix but n=1 per case and prose-only. **Both must be quoted with their date and context.** A re-run of the verification benchmark is required before publishing a detection rate.

D-06 — Blockchain-record check disagrees across artifacts. `benchmarks/verification-results.json` records `blockchainRecordPass: false` in all 10 Method A runs plus the chain-only probe (0/10). The Task 10 session records that this was caused by the public RPC's 10,000-block `eth_getLogs` cap and was fixed on 2026-10-16 by `resolveRegistrationTx`, after which Method A showed a real `BLOCK #50,315,729` and 6,198 confirmations. *Assessment:* the stored JSON is **pre-fix**. The paper must not quote 0/10 as a current behaviour, nor quote the fixed behaviour without noting the fix date.

D-07 — The issuance baseline was overwritten. Task 3 recorded an N=10 issuance baseline (means: fingerprint 4.67, pdf 14.67, ipfs 1524.84, prep 34.36, submit 1511.28, confirm 44972.59, supabase 456.29, total 48063.75 ms). The Task 10 session wrote a 1-run result into the same file, so `benchmarks/issuance-results.json` now holds **n = 1** and the N=10 raw data **no longer exists in the repository** (only the prose summary in `context_md/2026-10-14_231247.md`). *Assessment:* **the current issuance statistics are degenerate (n=1, `stdDev: 0` by convention, not by measurement)** and the old N=10 numbers are not comparable because they were produced by the pre-Task-4 code whose timeout path returned false successes. **Re-run with n ≥ 10 before publication.**

D-08 — Test-count progression: one session note disagrees with the file. `context_md/2026-10-15_235500.md:42,51-53` says the Method C file contains **16 tests** and struggles to reconcile 95 → 109. The actual file contains **14** `test(` occurrences, and 95 + 14 = 109 is consistent with the measured suite total. *Assessment:* the **file count (14) is authoritative**; the session note's "16" is an authoring-time miscount, self-flagged in the note. The user-supplied progression (83 → 85 → 95 → 109 → 123) is **correct** and is corroborated by `context_md` at every step (83 in the Task 4 note, 85 in Task 5, 95 in Task 6, 109 in Tasks 7/8, 123 in Tasks 9/10).

D-09 — Gas figures are stale and non-persisted. Recorded once (2026-10-14, 16-certificate era): registration mean 241,189 (n=16, 240,060–257,964), revocation mean 60,932 (n=4), ≈0.007236 POL at 30 gwei. No gas value appears in any benchmark JSON, and `eval-metrics.ts` writes nothing to disk. *Assessment:* **do not publish these as current results**; re-measure and persist.

D-10 — `eval-metrics.ts` prose results are stale. `context_md/2026-10-11_135113.md` records an 11-certificate registry; `2026-10-14_171533.md` records 16. The verification benchmark measured 28. *Assessment:* any `eval-metrics` number must be dated or re-measured.

D-11 — Historical security finding C-05 vs current state. `audit.md:102-112` describes a client-exposed `VITE_PINATA_JWT`. The current tree has `PINATA_JWT` (no `VITE_` prefix) read only in a server-only module, with client-bundle scans clean. *Assessment:* **the exposure is fixed in code**; whether the token was ever in a deployed bundle and whether it was rotated is **not recorded anywhere**. Flag as an open external action.

D-12 — Dashboard "Verified" metric semantics. `audit.md:200-207` warns the "Verified" metric counts certificates with an on-chain transaction hash rather than verification events. Confirmed still true at `dashboard.index.tsx:55`. *Assessment:* unchanged; label it honestly in any figure or claim.

D-13 — Evidence-form asymmetry. The three `benchmarks/*.json` files are the only persistent measurement artifacts. All live-experiment outcomes (Task 3/4/7/8/10, fingerprint-compat, revocation) exist only as prose in `context_md/`, and the Task 6 run logs were deliberately deleted. *Assessment:* reproducibility of live claims rests on session notes, not machine artifacts. Disclose.

D-14 — `.env.example` drift. `hardhat.config.cjs` reads `POLYGON_AMOY_RPC` and `POLYGONSCAN_API_KEY`; neither appears in `.env.example`. `.env.example` declares `VITE_CHAIN_ID` and `VITE_NETWORK_NAME`, which no code reads (the chain ID is hardcoded to 80002). *Assessment:* cosmetic but worth correcting before handover.

PAPER-READY FACT SHEET

The 34 points another AI must internalise before writing the paper.

1. Project: **CertiProof**, a blockchain-anchored academic certificate issuance and verification platform; final-year project; testnet only.
2. Deployed contract: `CertChainRegistry`, Solidity `^0.8.24`, MIT, **179 lines, no imports, no OpenZeppelin**, address `0x09921D25cB513793e17082Cce0fB4F520f0CDF50`, deployed **2026-08-31T06:04:59.550Z**, deployer/super-admin `0x3e72b748a543DEc762a297F7304A04F980d7bC01`.
3. Network: **Polygon Amoy testnet, chain ID 80002** (`0x13882`); explorer `https://amoy.polygonscan.com`; measured RPC `https://polygon-amoy-bor-rpc.publicnode.com`.
4. On-chain per certificate: id (mapping key + array), `certificateHash` (bytes32), `issuerAddress`, `ipfsCID`, `certificateNumber` (the app passes the id), `issueTimestamp`, `status` (0 active / 1 revoked). **Nothing else is anchored.**
5. Contract functions: 11 total; `registerCertificate` is `onlyAuthorizedIssuer`; `revokeCertificate` is `onlySuperAdmin` and **irreversible in code**; `superAdmin` can never change.
6. Events: `IssuerAdded`, `IssuerRemoved`, `CertificateRegistered` (3 indexed), `CertificateRevoked`.
7. **Fingerprint protocol:** `certificateId.trim() | recipientName.trim() | course.trim() | normalizeIssueDate(issueDate)` joined with `|`, SHA-256 over UTF-8 via WebCrypto, output `0x` + 64 **lowercase** hex (`/^0x[0-9a-f]{64}$/`).
8. `normalizeIssueDate`: trim → strip `T…`/` …` time suffix → zero-pad month/day → else keep verbatim. Internal whitespace is **significant**; only edges are trimmed.
9. The protocol is **byte-identical to the legacy formula**, so all pre-existing on-chain records keep their hashes — proven 24/24 stored and 24/24 chain MATCH.
10. **Three hashes, strictly separated:** metadataFingerprint (registry key), on-chain hash (the anchor it must equal), documentByteHash (SHA-256 of file bytes — reported, **never compared**). The only conflation in the codebase is in unimported dead code (`certiproof.ts`).
11. Issuance order: wrong-network guard → fingerprint → PDF (jsPDF) → IPFS pin → signer prep → `registerCertificate` (30 gwei priority / 50 gwei max fee, no explicit gas limit) → confirmation → certificate built from the **real receipt** → Supabase insert (non-blocking in app).
12. Supabase is written **only after a mined receipt** with `status 1`; a pending transaction produces **no** certificate, no store entry, and no database row.
13. **Confirmation semantics:** `confirmed` requires `status === 1` **and** `blockNumber > 0`; `failed` covers revert, unknown status, submission error, cancelled replacement; timeout yields `pending` and **never** success; `CONFIRMATION_TIMEOUT_MS = 60_000`; timers always cleared; no receipt is ever fabricated.
14. Pre-Task-4 defect (measured): 5/10 issuance runs reported success on timeout while mining **37–166 s later**.
15. Wrong-chain signing mines a **status-1 no-op** at an address without code; the guard runs **before** fingerprint/PDF/IPFS/signing; live-verified with only `eth_chainId` observed and **zero funds spent**.
16. **Method A** = format gate → read-only provider → linear registry scan → Supabase (gated) + event/receipt + issuer check + block number → 7 checks; outcome from hash match + `status` only.
17. **Method B** = QR payload `https://certiproof.app/verify/<fingerprint>` → regex extraction → identical to Method A. **No in-app camera scanning.**
18. **Method C** = byte hash → literal-string extraction with escape handling → label-anchored field reads with static-label rejection → two fingerprint recomputations (registry metadata and extracted document) → field-by-field comparison; `match` requires complete extraction **and** all field matches **and** both fingerprints equal to the chain hash.
19. Extracted fields: recipient (after *"this is to certify that"*), course (after *"has successfully completed"*), date (`Date:` + date-pattern validation), ID (`ID:` → `Certificate No:` → first `CERT-YYYY-NNNNNN`).
20. Missing fields return **explicit null**, never a placeholder; `complete` is true iff all four exist; incomplete extraction can never pass.
21. **Tamper evidence:** stored benchmark (pre-fix) **2/4 (50%)**; live post-fix **5/5** (recipient, course, date → `mismatch`; unknown ID → `not_found`). Quote both with dates. Scope is metadata tampering in CertiProof-generated **text** PDFs — **no OCR, no visual-tamper detection**.

22. **Verification performance (n=10 each):** A mean **6022.47 ms** (min 2707.02, median 6097.00, max 8006.71, stdDev 1678.63); B mean **6038.49 ms**; C mean **551.11 ms** (min 515.15, max 624.44, stdDev 32.66). A-miss (unknown hash) ~10.0–10.9 s; malformed hash ≤0.23 ms.
23. **Dominant latency:** `searchAndDecision` **≈ 99.9%** of Method A total; `registrySearch` **≈ 99.7%** of Method C total — the **linear O(n) registry scan** is the bottleneck (measured at a 28-certificate registry).
24. **Verification rates:** A 10/10, B-consistency 10/10, C 10/10, unknown→not_found 5/5, malformed→invalid_hash 4/4, fingerprint consistency **23/23** all-three-equal.
25. **IPFS (n=36 timed cycles + 3 dedup):** upload **39/39 (100%)**; retrieval first-attempt **36/36**; byte-identical integrity **36/36**; delayed recheck after **652,739 ms** still identical; same file → **same CID**, different file → different CID; error probes 4/4 with **no secret leakage**.
26. **IPFS latency:** upload overall mean **9508.25 ms**, median **3881.22**, min 1070.15, max 54534.18, stdDev 12037.31; retrieval overall mean **7181.63 ms**, median **7045.28**, min 4593.96, max 11009.05, stdDev 1687.95 — retrieval has a ~5–9 s floor independent of size.
27. **Issuance (n = 1, Task 10):** total **9328 ms**; fingerprint 1.74, pdf 33.55, ipfs 2117.72, prep 38.5, submit 1897.70, confirm 5232.65, supabase 1468.89 (excluded from total); QR microbenchmark 136.43 ms. Tx `0x823d7bc4…7803`, block **#50315729**, CID `QmVgzPmUdtTue47jgzwR1G3WogXuH9fgguoEgzRthmqoSu`.
28. **The N=10 issuance baseline was overwritten**; its raw data no longer exists in the repo (prose summary only: total mean 48063.75 ms under pre-Task-4 code). **Re-run before publishing issuance latency.**
29. **Revocation (live):** tx `0xfd8fa7f8…`, block **#50321993**, status 1; Method A `revoked`, B `revoked`, C `match` + status revoked; Supabase updated post-receipt with hash unchanged; `CERT-2026-273168` is now revoked on-chain.
30. **Supabase:** one table `certificates`, 12 columns, browser client with **public anon key**; SELECT ×3, INSERT ×1 (post-receipt), UPDATE (status only, chain-constrained), **DELETE never called**; **RLS is a proposal only and is NOT applied or verifiable from the repo.**
31. **Chain-authority protections:** metadata fingerprint gate, row reconciliation (drop non-anchoring, override status from chain, fail-open on RPC error), detail-page cross-check, chain-constrained status update (fail-closed), chain-first dashboard sync with chain fallback.
32. **Tests: 123 in 7 files, all offline/hermetic**, Bun test; progression 41 → 54 (Task 3) → 83 (Task 4) → 85 (Task 5) → 95 (Task 6) → 109 (Task 7/8) → **123** (Task 9/10). Gates `tsc --noEmit` and `vite build` green throughout. One recorded flaky run exited 1 with 0 failures (network flake in IPFS tests).
33. **Security posture:** Pinata JWT is server-only (client-bundle scans clean); deployer key confined to Node tooling with an empty fallback; **no auth on the upload function, no rate limiting, no CI, no contract tests, no dependency audit, no static analysis.**
34. **Honest framing:** the blockchain proves registration, issuer, status, and integrity of metadata — **not** academic legitimacy. The stack depends on a free public RPC, one pinning vendor, and a database with unapplied RLS. All results are testnet, small-n, single-environment measurements.

DO NOT INVENT

Everything below is currently **unknown, externally configured, insufficiently measured, or not experimentally proven**. None of it may be asserted, guessed, or filled in.

UNKNOWN / NOT IN THE REPOSITORY

- Live database schema and **whether RLS is enabled at all**.
- Whether the Pinata JWT was ever exposed in a deployed bundle, and whether it was rotated.
- The current on-chain registry state (counts of active/revoked) at the time of writing — every recorded count is dated.
- Whether the contract source is verified on Polygonscan.
- Any host/machine specification, runtime version, or commit hash for the benchmark runs.
- Whether the settings, email, and QR-bearing PDF download features persist or deliver anything (they are UI-only).
- Any academic statistic about credential fraud — none exists in this repo.

EXTERNALLY CONFIGURED DEPENDENCIES (CANNOT BE COMPLETED FROM THIS REPO)

- Applying `supabase/rls-hardening.sql` and verifying the live policies.
- Edge/CDN rate limiting and authentication on the IPFS upload function; Pinata quota alerts.
- A dedicated or archive RPC provider (needed for reliable `eth_getLogs` recovery).
- Dependency advisory audit and smart-contract static analysis (Slither).
- Deployment/hosting configuration (`cloudflare-module` preset exists; no committed deployment config).
- Issuer management tooling beyond `scripts/add-issuer.cjs`.

INSUFFICIENTLY MEASURED — DO NOT PUBLISH AS RESULTS

- **Issuance latency distribution** (currently n = 1; the N=10 baseline is gone).
- **Tamper-detection rate** (stored 2/4 pre-fix vs live 5/5 post-fix, n=1 per case; needs a re-run).
- **Gas cost** (stale 16-certificate-era numbers, prose only, not persisted).
- **Scalability / latency growth with registry size** (O(n) is a code observation; only k∈{1..16} extrapolation exists, in prose).
- **IPFS reliability beyond one day / one gateway / 36 cycles.**
- **Anything about mainnet**, cost comparisons, or economic advantage.
- **Usability, adoption, or user-satisfaction** — no study exists.
- **Contract-level behaviour under adversarial conditions** — no contract test suite exists.

CLAIMS THAT MUST NOT BE MADE

- ”100% secure”, “completely decentralized”, “impossible to forge”, “100% reliable”, “zero vulnerabilities”, “production ready”, “better than all existing systems”, or any unqualified “100%” success claim.
- That the blockchain proves the academic legitimacy of a credential.
- That CertiProof detects arbitrary visual document forgery.
- That verification is fast (≈6 s mean at n≈28, dominated by a linear scan).
- That the system has no tests (123 exist) — and equally, that it is fully tested (no contract tests, no CI).
- That the repository’s own `PROJECT_SUMMARY.md` describes the current system (it describes the discarded mock era).

End of report. Generated 2026-10-16 from the repository working tree and `context_md/` session history. No code, configuration, or on-chain state was modified in producing it.